

Cost and Precision of Spotlight SAR Image Formation on Google TPUs, an Nvidia GPU and a CPU

Paul Singerman
Penn State University
pgs16@psu.edu

Tom Braun
Penn State University
trb5864@psu.edu

October 2026

Abstract

Spotlight SAR image formation was implemented for Google TPUs, an Nvidia GPU and a CPU and measured on three Umbra collections at native size. Factorized backprojection runs in kernels written for each device (Pallas, CUDA, C++); images are scored against a float64 exact backprojection checked against the vendor’s image. At float32 accuracy a thousand Panama images cost \$0.76 on the L4, \$1.62 on a TPU v5e, \$1.89 on a v6e and 4.3 times the L4’s figure on the CPU. The v6e is fastest, 1.5 times the L4’s speed at 3.8 times its hourly price. Float16 on the L4 adds 0.9 dB of error and saves 19%; the TPUs’ native single-pass products add 12 dB, mostly as phase error beside bright returns. Polar format is the cheapest algorithm on the L4 but runs 47 to 51 times slower on a TPU, and its image carries an unexplained -32 dB residual.

1 Introduction

A spotlight SAR processor now has a choice of hardware it did not have five years ago. The cloud rents GPUs and TPUs by the hour at prices set by the machine-learning market, and a phase history of 2×10^8 complex samples forming 10^8 pixels is, on paper, the regular arithmetic those devices were built for.

The difficulty is that they were built for different arithmetic. A GPU is fast on the operation backprojection performs, an independent per-pixel loop that reads a range profile at a computed address. A TPU does that operation on its vector unit at a rate two orders of magnitude lower, and is fast only when the work is a dense matrix product with bfloat16 operands. The question for a shop deciding where to put its quota is therefore not which device has the higher peak rate. It is whether image formation can be rewritten in the TPU’s idiom without losing accuracy, and what the rewritten algorithm then costs against a GPU running the natural one.

That question was put to three Umbra collections at the vendor’s native size, formed on four instances by three algorithms and scored against a float64 exact backprojection. The answer is that the L4 forms the float32-accuracy image for \$0.76 per thousand against \$1.62 and \$1.89 on the v5e and v6e, with kernels written for each device and taken to measured fractions of each device’s limits (Appendix D), and that the CPU costs 4.3 times the L4 (Table 3). The v6e is the fastest device. What separates the accelerators beyond that is how each behaves when the arithmetic is lowered (Figure 1), and polar format is the cheapest algorithm on the GPU and the slowest on a TPU (Table 3, Appendix E). An earlier internal round on smaller, simulated data is superseded here and is referred to below as the first round.

Figures 2 to 4 show the images: the three scenes, four Panama regions across devices and algorithms with their coherence against the reference, and three regions at pixel scale (0.41 by

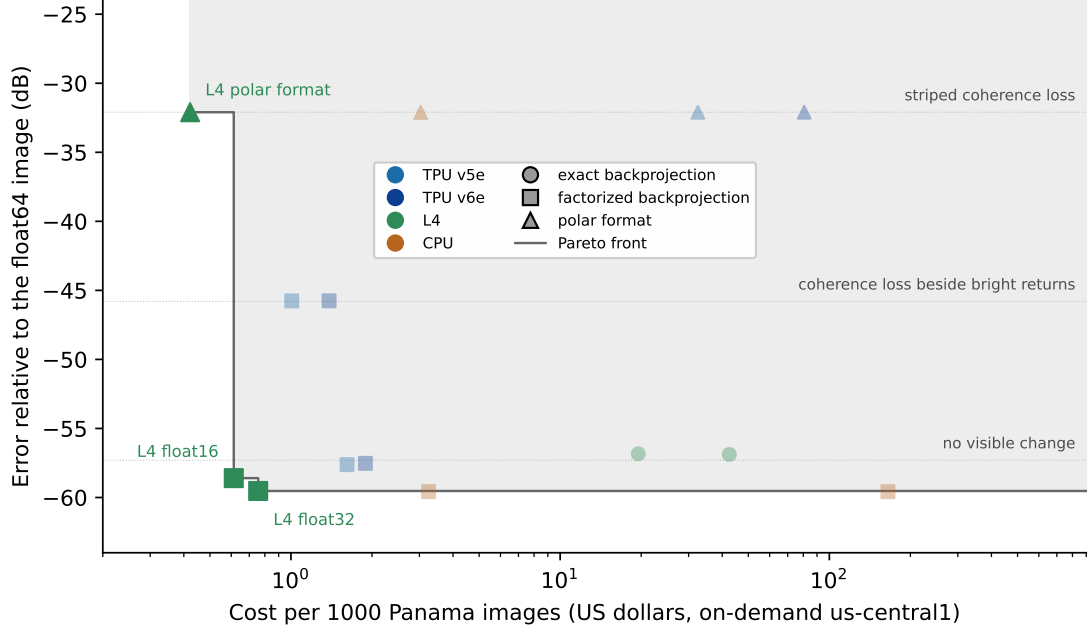


Figure 1: Cost per thousand Panama Canal images against the error of the image relative to the float64 reference, for the selected configuration of each device, algorithm and precision (Table 3). Cost and error are defined in Section 5. The line is the Pareto front, the 3 configurations that no other configuration beats on both cost and error (errors within 0.05 dB counted as equal); only these are labeled, and the dominated configurations are drawn faded. The dotted lines mark the three regimes of visible change described in Section 6.

0.36 m). In amplitude no image differs visibly from the reference; the differences are in the complex values, where the coherence maps show them.



Figure 2: The three collections as formed by the float64 reference, downsampled for display (amplitude in decibels over a 45 dB range, azimuth horizontal, range vertical, 1 km bar). The yellow boxes on the Panama Canal image are the four 512 by 512 pixel regions of Figure 3.

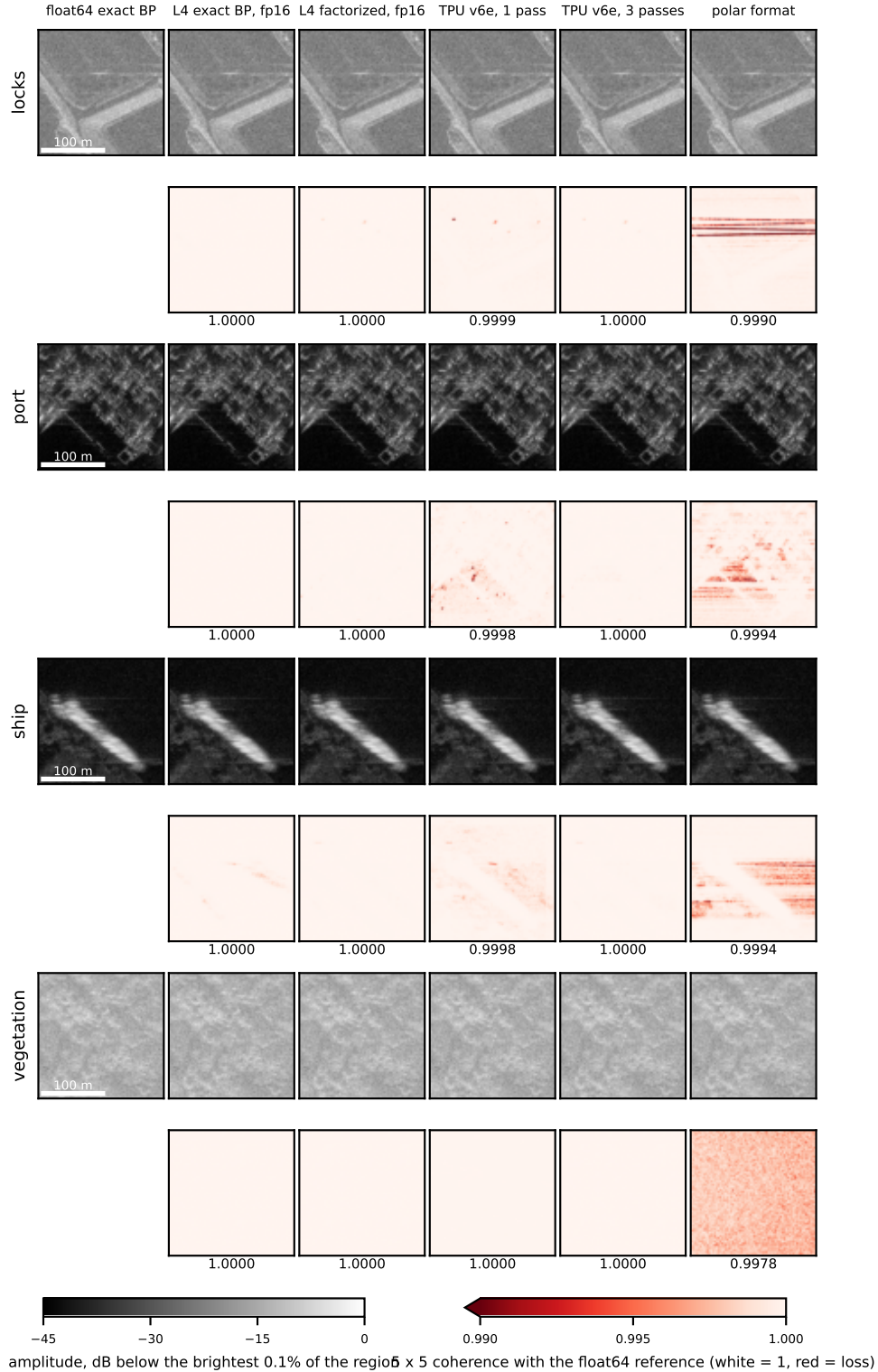


Figure 3: Panama Canal, four regions of 512 by 512 pixels at full resolution (100 m bar). Top of each pair: amplitude over a 45 dB range for the float64 reference, the L4 exact backprojection (float16 profiles), the L4 factorized image in float16 (CUDA kernels), the TPU v6e factorized image with single-pass and three-pass products, and polar format. Bottom: the 5 by 5 coherence of each image with the reference from 0.99 (red) to 1 (white), with its mean below each panel. The regions are the Cocolí Locks, the Port of Balboa, a ship in the channel and vegetated terrain 2.7 km from the scene center; Appendix I locates them.

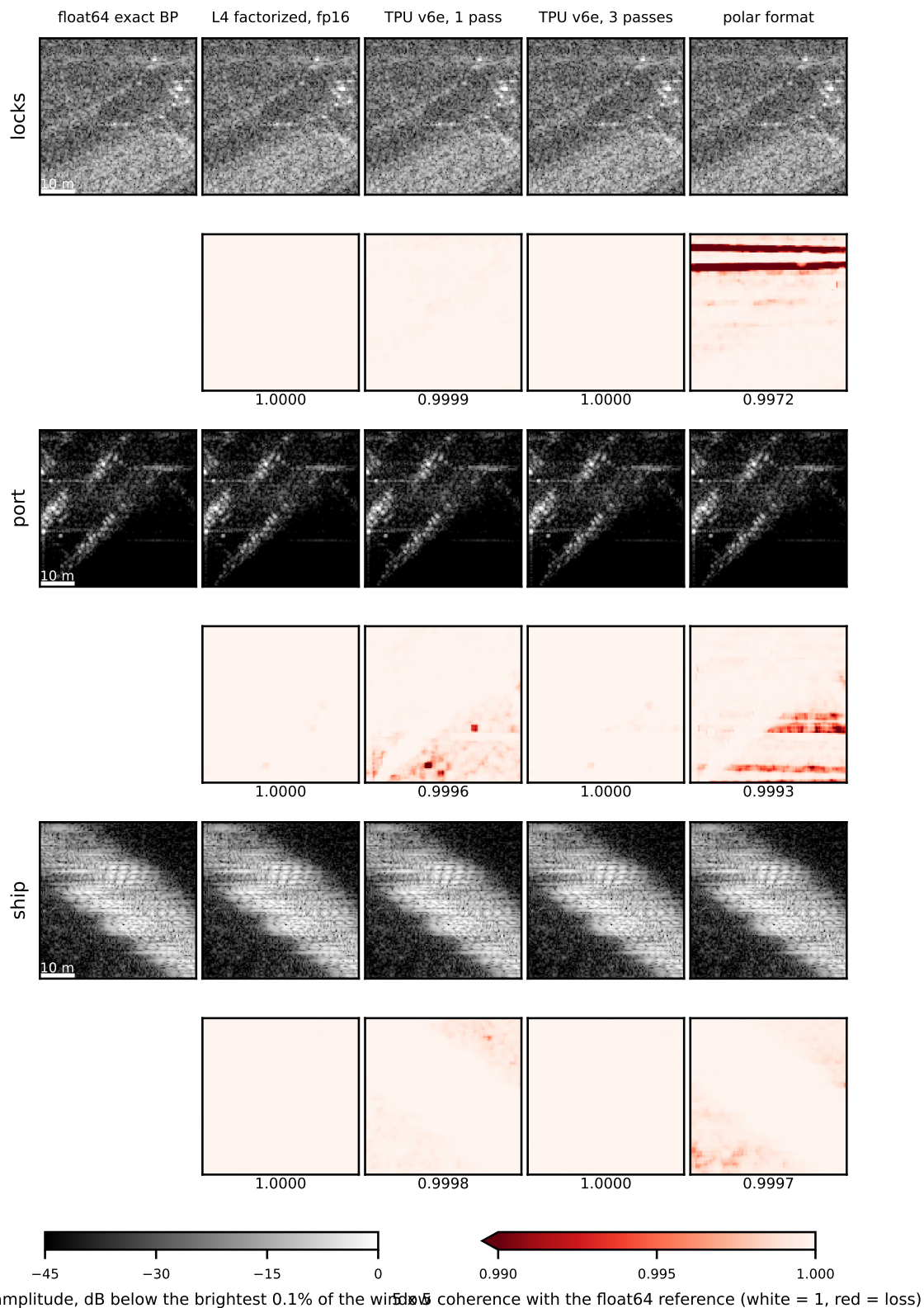


Figure 4: Panama Canal at pixel scale: 128 by 128 pixel windows (53 m in azimuth by 46 m in slant range, 10 m bar) around the brightest feature of the lock, port and ship regions of Figure 3, drawn without interpolation so that each pixel (0.41 by 0.36 m) is visible on the page. Columns, scales and coherence maps as in Figure 3, without the L4 exact backprojection, which does not differ visibly from the reference at this scale either.

2 Prior work

Backprojection on parallel hardware has a fifteen-year literature, almost all of it on GPUs and almost none of it on cost. Fasih and Hartley gave the first single-GPU CUDA implementation [4]; Benson, Campbell and Cook scaled it to gigapixel images on a GPU cluster and reported throughput as a function of precision [5]; Park and colleagues reached 35 billion backprojections per second per node on Xeon and Xeon Phi processors by strength reduction and locality optimization [6]; Cordes and Leiser ran the kernel on FPGAs with general-purpose hosts [7]. On the accuracy side, Capozzoli, Curcio and Lisenò replaced the linear interpolation of the range profiles with a NUFFT and showed it more accurate at similar cost on a GPU [8], and Xu and colleagues recently ran a streaming GPU backprojection in float16 [9]. These papers establish that the GPU is the natural home of the exact algorithm; none measures what an image costs in dollars, and none compares against a device that cannot gather. Open-source backprojection implementations exist for the CPU (RITSAR [24], the AFRL MATLAB toolbox of Gorham and Moore [27], ISCE3 [25]) and the GPU (torchbp [26], and a CUDA version of the ISCE3 backprojection). Appendix G compares them on identical inputs with FastSAR, the open-source release of the kernels evaluated here.

The two approximate algorithms used here are older. Fast factorized backprojection descends from McCorkle’s quadtree [28] and Ulander’s formulation [30], was applied to strongly nonlinear airborne tracks by Frey and colleagues [10], and was put on GPUs for circular SAR by Ponce and colleagues [11]. Polar format is the textbook algorithm [31, 32]; its scene-size limit from wavefront curvature was derived by Doerry together with a post-processing compensation [12], and derived again in terms of the residual quadratic phase error by Gorham and Rigling [13]. Jakowatz and Doren compared polar format with backprojection for spotlight images in 2006 [14]. The polar format of this report follows that line: the image is formed fast and the known distortion is removed in a final resampling.

The TPU has been used outside machine learning only where a problem can be written as dense matrix products. Jouppi and colleagues describe the architecture and its matrix units [15], and the fourth generation’s sparse cores for embedding lookups [20]. Lu and colleagues formulated large discrete Fourier transforms as Kronecker products of dense matrices on TPU clusters [21] and then the nonuniform FFT, reporting 20 to 80 times a CPU on one TPU card [22]; Lewis and colleagues ran distributed dense linear algebra at supercomputer scale on TPU pods [23]. Those results are the reason the factorized algorithm here is written entirely as matrix products. What the literature does not contain is a measurement of SAR image formation on a TPU at all, or a cost comparison between TPU, GPU and CPU on real spotlight collections at native size against a common float64 reference.

3 Data

Three collections from Umbra’s open data catalog [1] were used (Table 1). They are the Pacific entrance of the Panama Canal (2023-07-18, 02:30 UTC), with the Cocolí Locks, the navigation channel and the Port of Balboa; central Melbourne (2023-02-08); and farmland near Johnston, Iowa (2023-10-20). Each was processed from the compensated phase history (CPHD) as delivered, apart from 43 leading pulses of the Melbourne collection that carry no positions or samples. The image grid is the vendor’s own: the pixel counts, spacings and slant image plane of the vendor’s complex image (SICD).

The four Panama regions of Figure 3 were identified against Sentinel-2 imagery and six other Umbra passes, which also show that the bright target of the ship region is a ship moving at 3.0 m/s,

Table 1: The three collections and their native image grids. Spacings are azimuth by range in the slant plane; squint is the Doppler cone angle less 90° at the aperture center; range is the slant range to the scene center at the start of the aperture.

Scene	Image (az $\times$ rg)	Spacing (m)	Pulses $\times$ samples	Bandwidth (MHz)	Graze	Squint	Range (km)
Panama Canal	$12,207 \times 8,808$	0.413×0.356	$15,186 \times 14,399$	337	42.6°	-28.1°	738
Melbourne	$10,077 \times 10,273$	0.422×0.298	$15,948 \times 14,399$	405	43.4°	0.6°	760
Iowa farmland	$8,521 \times 8,465$	0.517×0.328	$13,003 \times 11,663$	356	50.3°	2.8°	653

Table 2: Instances, the arithmetic timed on each (Section 4 and Appendix B) and on-demand prices (US dollars per hour, us-central1).

Label	Instance	Arithmetic timed	Price
TPU v5e	1 chip (v5litepod-1), 24 vCPU host	bfloat16 single-, three- and six-pass products; float32 or bfloat16 data	1.200
TPU v6e	1 chip (v6e-1), 44 vCPU host	bfloat16 single-, three- and six-pass products; float32 or bfloat16 data	2.700
L4	g2-standard-4, 1 Nvidia L4 (24 GB)	float16, TF32 and float32 products; float16 or float32 data	0.707
CPU	c4d-highmem-16, AMD EPYC 9B45, 8 cores, 126 GB	float32 and float64	0.964

imaged 194 m from its position (Appendix I).

4 Devices, algorithms and arithmetic

Table 2 lists the instances and their on-demand us-central1 list prices (Google Cloud Billing Catalog, 1 October 2026). The CPU instance is the smallest that holds the float64 factorized image; the L4 instance is the smallest offered with that GPU. Software was JAX 0.11.2 throughout, with CUDA 12 and CuPy 14.2.0 on the L4 and Numba 0.68.0 for the reference.

Three algorithms were run (Appendix A). *Exact backprojection* sums, for every pixel, the range-compressed pulse interpolated at the pixel’s range [27]: one indexed read per pixel and pulse, 1.6 trillion for the Panama image. A CUDA kernel runs it on the L4 with 8 times oversampled profiles in float32 or float16; a float64 CPU version with 16 times oversampling is the reference. On the TPU the same gather runs at 4×10^7 to 10^8 reads per second depending on the access pattern, against about 10^{10} on the L4 (Appendix C), so exact backprojection was not run there.

Factorized backprojection divides the image into tiles, re-centers the phase history on each tile with a phase ramp, filters and decimates it, and recurses [28, 30]. Every stage is an element-wise or a dense matrix product, the operation the TPU’s matrix units are built for. One JAX program runs on all four devices. It is the baseline for the kernels, and it forms every polar-format image and the float64 factorized image. Where it is faster, strided convolutions replace the decimation matrices (*conv. filters* in the tables), and the phase ramps are evaluated directly or from a coarse and a fine table (*direct* and *split-table* ramps).

The factorized rows of Table 3 use kernels written for each device, Pallas on the TPUs, CUDA on the L4 and C++ with OpenMP on the CPU (*fused kernels*, *CUDA kernels* and *C++ kernels* in the tables), in which the rotated phase history of each child and the ramp matrices of the final stage stay in on-chip memory (vector memory on the TPUs, shared memory on the L4, cache on the CPU) instead of passing through main memory. Appendix D gives the kernels, the builds that led to them and their bounds against the units of each device; the CUDA and C++ kernels evaluate the geometry of every level in float64, so their float32 images are 1.6 to 2.2 dB closer to the reference than those of the JAX program, which evaluates the last level’s geometry in float32; the TPU kernels keep the JAX program’s geometry, and their images agree with its images to within their

rounding. The tiles’ band limits leave a residual against exact backprojection that is independent of the arithmetic (Table 13, float64 row).

Polar format resamples the pulses and samples onto a rectangular wavenumber grid by chirp-z transforms [33] and forms the image by one FFT [31, 32]; it is the cheapest of the three (Table 3). It assumes a planar wavefront, so scatterers are displaced by an amount that grows with the square of their distance from the scene center; a final resampling maps the image through the fitted displacement (Appendix E). Its pulse resampling runs as a dense product or as a sum over gathered rows (*gather*).

The TPU’s matrix units multiply bfloat16 operands only. A float32 product is formed from one bfloat16 pass by default, which gives bfloat16 accuracy, or from three or six passes (JAX precision *default*, *high*, *highest*), called single-, three- and six-pass products below. The L4’s tensor cores take float16 operands (11 significant bits against bfloat16’s 8), so *sixteen-bit* means a more accurate type on the L4 than on the TPU. The L4’s default float32 product uses TF32 operands (also 11 bits); the configurations called *float32* on the L4 use full float32. Every configuration keeps the geometry in float32 or float64 and lowers only the storage type of the data and the operand type of the products; the CPU runs native float32 and float64 (Appendix B).

5 Measurement protocol

Device time per image is the wall time of one image on a warm device with the data resident, compilation excluded and the per-collection host work included (about 0.5 s). Throughput is the number of images per hour of a pipelined loop in which the transfers of the next collection and the previous image overlap the formation of the current one. Cost per image is the instance price divided by the throughput; where no pipelined loop was run, the inverse device time is used and the entry carries an asterisk. Repeated timings agree within 4%, and the L4’s pipelined factorized loops run 6 to 11% slower per image than its device time (Appendix H).

Every image is compared with the float64 exact backprojection of the same collection after one gain has been fitted per algorithm family (complex for the backprojections, real for polar format); the reference itself is registered to the vendor’s own image of each collection to a fraction of a pixel (Appendix F). The error quoted is the energy of the difference relative to the energy of the reference (dB). The largest pixel difference, the 5 by 5 coherence and the amplitude and phase deviations over the brighter half of the pixels are tabulated with it. The selected configuration of each device, algorithm and precision is the fastest implementation form whose image is within 0.1 dB of the other forms on that device. Power was measured on the L4 only, from `nvidia-smi`.

6 Results

6.1 Time and cost per image

A single error figure in decibels does not tell an analyst whether an image is usable, because a 10 dB change in the residual can be invisible in amplitude and obvious in a coherence product. The configurations fall into three regimes of what a viewer would notice against the float64 image, with thresholds taken from the statistics of Table 13 (the dotted lines of Figure 1). In the first there is no visible change: the 99th-percentile amplitude deviation is within 3 dB and the 5 by 5 coherence stays above 0.999 even at its 0.1 percentile. In the second the amplitude image is the same but coherence falls below 0.999 at the darkest tenth of a percent of pixels, which Figures 3 and 4 show as a halo beside bright returns. In the third coherence falls below 0.99 in stripes.

Table 3: Panama Canal: the selected configurations, cheapest first. Bold rows are the Pareto front of Figure 1, the configurations that no other configuration beats on both cost and error (errors within 0.05 dB counted as equal). Cost and error as defined in Section 5; an asterisk marks rows timed without a pipelined loop, with cost computed from the device time. Energy is that of the processor alone over the pipelined per-image time that the cost column uses. On the L4 it is measured by `nvidia-smi`. On the TPUs it is a range from the design-power figures third parties quote (120 to 200 W for the v5e, 200 to 350 W for the v6e [18, 19]), since Google publishes none [17], applied as if the chip ran at that power throughout. On the CPU it is the 8 cores’ pro-rata share of the 400 W package times their occupancy, taken as the busy fraction of the 16 hardware threads doubled and capped at one. Throughput and the other two collections are in Table 14.

Device	Algorithm	Arithmetic	Time (s)	\$ per 1000	kWh per 1000	Error (dB)
L4	Polar format	float32, gather	2.1	0.42	0.04	−32.1
L4	Factorized BP	float16 throughout, CUDA kernels	2.9	0.61	0.05	−58.6
L4	Factorized BP	float32, CUDA kernels	3.6	0.76	0.07	−59.5
TPU v5e	Factorized BP	single-pass products, fused kernels	2.8	1.01	0.10 to 0.17	−45.8
TPU v6e	Factorized BP	single-pass products, fused kernels	1.7	1.39	0.10 to 0.18	−45.8
TPU v5e	Factorized BP	three-pass products, fused kernels	4.6	1.62	0.16 to 0.27	−57.6
TPU v6e	Factorized BP	three-pass products, fused kernels	2.4	1.89	0.14 to 0.25	−57.5
CPU	Polar format	float32, gather	11.3	3.03*	0.08	−32.1
CPU	Factorized BP	float32, C++ kernels	12.4	3.25	0.08	−59.6
L4	Exact BP	float16 profiles	103.8	19.50	1.97	−56.8
TPU v5e	Polar format	float32, gather	97.4	32.48*	3.25 to 5.41	−32.1
L4	Exact BP	float32 profiles	216.4	42.49	4.31	−56.9
TPU v6e	Polar format	float32	105.3	80.66	5.97 to 10.46	−32.1
CPU	Factorized BP	float64, conv. filters, direct ramps	616.3	165.03*	2.84	−59.6

The spread inside the first regime is large, \$0.61 to \$165 per thousand, and almost all of it is exact backprojection and the CPU. The TPUs’ native arithmetic opens the second at \$1.01 on the v5e, and polar format on the L4 the third at \$0.42. Bold in Table 3 marks the 3 configurations on the Pareto front of Figure 1, the ones nothing else beats on both cost and error.

The v6e forms the factorized image fastest, in 1.7 s with single-pass products and 2.4 s with three-pass products (Table 3; other collections in Table 14, every timed configuration in Appendix J). The v5e takes 2.8 and 4.6 s for the same two, and the L4 3.6 s in float32 and 2.9 s in float16. The hourly price rises faster than the device speed: the v6e is 3.8 times the L4’s price for 1.5 times its speed.

At float32 accuracy a thousand Panama images therefore cost \$0.76 on the L4, \$1.62 on the v5e and \$1.89 on the v6e. The order holds on every collection (Table 14): on Melbourne the three figures (L4, v5e, v6e) are \$0.78, \$1.65 and \$1.73, on Iowa \$0.50, \$1.07 and \$1.26. With its C++ kernels on 8 cores, the CPU forms the factorized Panama image in 12.4 s at \$3.25 per thousand images, 4.3 times the L4’s cost; the JAX program takes 217.4 s on the same instance (Appendix D).

With the sixteen-bit arithmetic each device offers, the L4 costs \$0.61 per thousand (float16 throughout: phase history, intermediates and final-stage operands in float16, accumulation in float32), and the v5e and v6e cost \$1.01 and \$1.39 with single-pass bfloat16 products. The L4 float16 image and the TPU single-pass images differ by 12.8 dB in error (Section 6.2); the first shows no visible change, the TPU images lose coherence beside bright returns. A third L4 setting, float32 storage with only the final stage in float16 on the tensor cores, sits between the two L4 rows in time and in error (Appendix D). Exact backprojection on the L4 costs \$19.50 per thousand with float16 profiles and \$42.49 with float32, 32 and 56 times the cost of the factorized image of the same

storage precision on the same card.

6.2 Precision

Table 13 (Appendix J) gives the error statistics of the Panama configurations; Figure 1 plots their error against cost. The factorized errors on Melbourne and Iowa are within 1.6 dB of the Panama values. The reference’s own convergence, measured on a crop at 8, 16 and 32 times oversampling, is -69.2 dB at 16 times, so no exact-backprojection error can be measured below about -69 dB. Exact backprojection on the L4 with 8 times oversampled profiles has an error of -56.9 dB with float32 profiles and -56.8 dB with float16; the two coincide because the interpolation error exceeds the arithmetic error at this oversampling. With 16 times profiles the images reach -78.5 and -74.5 dB against the 16 times reference whose interpolation they share, and about -69 dB in truth, at 203.3 s per image with float16 profiles (Appendix H).

Factorized backprojection at float32 accuracy has an error of -57.5 dB on the v6e with three-pass products, -59.5 dB on the L4 and -59.6 dB on the CPU. The float64 image, whose residual is the band limitation of the tiles, is at -59.6 dB; the float32 images of the L4 and CPU kernels are within 0.1 dB of it and the three-pass TPU image is 2.1 dB above it; the JAX program on the CPU, which evaluates the last level’s geometry in float32, gives an image 1.6 dB above it. On Panama and Melbourne, exact backprojection at 8 times oversampling is 2.2 to 2.7 dB farther from the reference than the L4’s float32 factorized image and takes 29 to 69 times its device time (float16 and float32 profiles). On the smaller Iowa collection it is about 0.4 dB closer. The L4’s float16-throughout image, at -58.6 dB, is 0.9 dB above its float32 image; keeping the levels in float32 and lowering only the final stage adds 0.2 dB (Appendix J).

The TPU’s single-pass products round both operands to bfloat16 and give an error of -45.8 dB, which matches the rounding noise of two operands accumulated through three levels (Appendix B). The 99th-percentile amplitude and phase deviations are 0.16 dB and 1.0° ; the extremes are 10.1 dB and 86° (172° on Melbourne), at isolated pixels beside bright returns, where Figures 3 and 4 show local losses of coherence. These differences are confined to the complex values and would be expected to matter for interferograms and change-detection maps, which were not tested here.

Three-pass products reproduce the six-pass image to within 0.3 dB in every decibel statistic and 0.003 in coherence. They take 1.4 (v6e) and 1.7 (v5e) times the single-pass time, well below three, because the matrix-unit products are only part of each kernel’s time: on the v6e they bind only the final stage, and on the v5e, where they bind most stages, the vector work and memory traffic run alongside them (Table 7). Bfloat16 storage of the data changes none of these figures and was slower on the two larger collections. The L4’s float16-product image has no TPU equivalent: the TPU converts float16 operands to bfloat16 before the matrix units (Appendix B).

6.3 Power

The L4 forms a float32 factorized Panama image using 0.23 kJ of GPU energy and a float16 image using 0.18 kJ (Table 4), drawing 64 W in the kernel loops against its 72 W limit; on the pipelined per-image time that is 0.07 and 0.05 kWh per thousand images (Table 3). The TPU entries are ranges built from the design-power figures third parties quote, since Google publishes none: 0.16 to 0.27 kWh per thousand on the v5e and 0.14 to 0.25 on the v6e with three-pass products, with lower ends about twice the L4 figure. The CPU estimate is 0.08 kWh. None of the entries includes the host, and the TPU and CPU figures are estimates, not measurements.

Table 4: Energy per Panama image for the processor alone. The L4 draw is the mean of `nvidia-smi` samples during the pipelined loop; the TPU figures are bounds from the design-power figures third parties quote [18, 19], since Google publishes none, assuming the chip ran at that power throughout; the CPU figure is the pro-rata share of the 8 cores (25 W of the 400 W package) times their occupancy, the busy fraction of the 16 hardware threads doubled and capped at one. Hosts, memory and the other components of each instance are excluded.

Device	Configuration	Basis	Power (W)	Time (s)	Energy (kJ)
L4	Factorized BP, float16 throughout, CUDA kernels	measured	63	2.9	0.18
L4	Factorized BP, float32, CUDA kernels	measured	64	3.6	0.23
L4	Exact BP, float16 profiles	measured	71	103.8	7.42
TPU v5e	Factorized BP, single-pass, fused kernels	design-power bound	120 to 200	2.8	0.3 to 0.6
TPU v5e	Factorized BP, three-pass, fused kernels	design-power bound	120 to 200	4.6	0.5 to 0.9
TPU v6e	Factorized BP, single-pass, fused kernels	design-power bound	200 to 350	1.7	0.3 to 0.6
TPU v6e	Factorized BP, three-pass, fused kernels	design-power bound	200 to 350	2.4	0.5 to 0.8
CPU	Factorized BP, float32, C++ kernels	pro-rata estimate	25 at 95%	12.4	0.29

7 Fairness of the comparison

The device comparison depends on each device having been run with an algorithm and arithmetic appropriate to it; Appendices C and H give the measurements behind the following. On the TPU, a profile of the first factorized level of the JAX program showed the two decimation products taking 4.6 ms per tile and the phase rotation before them 9.8 ms, with the rotation moving 534 GB/s: the device time was set by vector-unit work and memory traffic rather than by the matrix units. In the kernels that replaced that program the vector unit still binds the v6e’s first two levels, while on the v5e, whose matrix units are slower, they bind most stages (Table 7). In the JAX program, padding to multiples of 256 (8.3 to 8.9 s), split-table ramps (4.5 against 3.8 s), bfloat16 storage (5.2 against 4.5 s), block-banded products and strided convolutions (the latter four times slower) were each tested and none was faster.

On the L4 the exact kernel is dedicated CUDA code with float64 tile geometry and the factorized rows are the CUDA kernels of Appendix D. On the CPU the factorized float32 row is the C++ kernel build of Appendix D, with one OpenMP thread per core and 95% of the 8 cores busy. The polar-format and float64 rows come from the JAX program, timed as a single process. On the JAX float32 program, the baseline of Appendix D, one process kept 65% of the 16 hardware threads busy and four pinned processes 89%. For the factorized algorithm each device has kernels whose stages run within 1.0 to 2.5 times the bound set by the unit that binds them (Table 7); the improvements still identified are about 12% on the TPUs and 8% on the L4, and on the CPU, whose stages run at 1.05 to 1.4 times their bounds, none was quantified (Appendix D). The polar-format and exact algorithms were not optimized to this standard, so their rows are upper bounds on cost. The factorized plan (32-pixel final tiles, three levels, a passband fraction of 0.4 and 70 dB filter attenuation) kept its defaults on every device, and a different factorization would move the bounds themselves by an amount this study does not measure. Five further asymmetries apply. The TPU7x and H100 were unavailable under the project’s quota; the final resampling of polar format was written once, as a gather; the asterisked rows of Table 3 are inverse device times; the TPU hosts (24 and 44 vCPUs) are larger than the work needs while the L4’s (4 vCPUs) is the smallest offered; and spot and committed prices were not considered.

8 Conclusions

The conclusions hold for the fastest form of each flow found in this study (Section 7 and Appendix D).

1. For routine factorized imaging at float32 accuracy, the L4 is the cheapest of the four machines. A thousand Panama images cost \$0.76 on the L4, \$1.62 on the v5e and \$1.89 on the v6e, and the order is the same on Melbourne and Iowa (Table 14). The CPU, with C++ kernels whose stages run within 1.05 to 1.4 times their bounds, costs \$3.25 per thousand, 4.3 times as much as the L4. The L4 forms the image in 3.6 s, 1.5 times the v6e’s time, at 0.26 of its hourly price.
2. The v6e is the choice when latency matters. It forms the Panama image in 2.4 s with three-pass products, 1.5 times faster than the L4 in float32 at 3.8 times the hourly price, and it is not on the Pareto front at any precision. The v5e forms each of the same images at the same error, at lower cost and in more time.
3. The saving from sixteen-bit arithmetic, and the error it introduces, differ by device and by product. On the L4, float16 throughout saves 19% of the cost and adds 0.9 dB of error, with the amplitude image, the coherence and the phase statistics unchanged to the precision of Table 13; on that card it should be the default for every product. On the TPUs, single-pass products save 27% (v6e) and 38% (v5e) against three-pass and add 12 dB of error. The amplitude image is unchanged and the mean coherence stays at 0.9999; the lowest coherence falls to 0.705 (against 0.886 with three-pass products, the same as the float64 image’s) beside bright returns, where the phase error reaches 172° (Melbourne; 86° on Panama). For an amplitude product (detection, mapping, visual interpretation) single-pass is the appropriate TPU setting, and the single-pass TPU rows are then the ones to compare with the L4’s: \$1.39 (v6e) and \$1.01 (v5e) against \$0.61. For a coherent product (interferometry, coherent change detection) the phase error beside bright returns is the quantity of interest, and three-pass products, at 1.4 to 1.7 times the single-pass time, remain necessary. Storing the data in bfloat16 on the TPUs gave no further saving.
4. Exact backprojection at 8 times profile oversampling, often taken as the standard, gives no advantage here: on two of three collections it was no more accurate than the float32 factorized image and cost 26 to 63 times as much as that image on the same card. Its accuracy is limited by the profile interpolation, and only at 16 times oversampling does it become the most accurate image in the study (about -69 dB), at 203.3 s per image with float16 profiles.
5. Polar format is the cheapest algorithm on the L4 when an error near -32 dB, 27 dB above the factorized image, is acceptable. It registers with backprojection and agrees with it to -32.1 dB; on the L4 that image costs \$0.42 per thousand, 1.8 times less than factorized backprojection on the same card after the kernel work (7 times before it). On the CPU, where polar format runs only as the JAX program, its image costs \$3.03 per thousand against \$3.25 for the factorized C++ kernels. On the TPUs the scattered reads of its final resampling alone take 42 to 104 s, longer than the whole factorized image, so a TPU should not be used for polar format until that step is rewritten without gathers or moved to the host. The residual, which grows from -40 dB at the center to -30 dB in the outer quarter, remains unexplained (Appendix E). It does not affect amplitude products; for interferometric and change-detection products it sets the error floor of the polar-format image.

Acknowledgments. The code and the text of this report were drafted with the assistance of a large language model; the authors designed the study, ran and checked every measurement, and take full responsibility for the content.

Code and data. The code, the cloud run scripts, every measurement record behind the tables and the build of this report are at <https://github.com/saulpingerman/sar-accel-study>. The radar data are from the Umbra Open Data Program [1] and the optical images from Copernicus Sentinel-2 [3]; neither is redistributed, and the scripts fetch both from their public sources. The image-formation kernels are also packaged on their own, behind one call for the TPU, GPU and CPU backends, as FastSAR (<https://github.com/saulpingerman/FastSAR>).

References

- [1] Umbra Lab, Inc., “Umbra Open Data Program,” 2023. [Online]. Available: <https://umbra.space/open-data>
- [2] R. K. Raney, “Synthetic aperture imaging radar and moving targets,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. AES-7, no. 3, pp. 499–505, 1971.
- [3] European Space Agency, “Copernicus Sentinel-2 Level-2A surface reflectance, tiles S2B_17PPK_20230723 and S2A_17PPK_20240223,” contains modified Copernicus Sentinel data 2023 and 2024, accessed October 2026 through the Earth Search STAC catalog. [Online]. Available: <https://registry.opendata.aws/sentinel-2-l2a-cogs/>
- [4] A. R. Fasih and T. D. R. Hartley, “GPU-accelerated synthetic aperture radar backprojection in CUDA,” in *Proc. IEEE Radar Conf.*, Washington, DC, 2010, pp. 1408–1413.
- [5] T. M. Benson, D. P. Campbell, and D. A. Cook, “Gigapixel spotlight synthetic aperture radar backprojection using clusters of GPUs and CUDA,” in *Proc. IEEE Radar Conf.*, 2012, pp. 853–858.
- [6] J. Park, P. T. P. Tang, M. Smelyanskiy, D. Kim, and T. Benson, “Efficient backprojection-based synthetic aperture radar computation with many-core processors,” in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC ’12)*, Salt Lake City, UT, 2012.
- [7] B. Cordes and M. Leeser, “Parallel backprojection: A case study in high-performance reconfigurable computing,” *EURASIP J. Embedded Syst.*, vol. 2009, art. 727965, 2009.
- [8] A. Capozzoli, C. Curcio, and A. Liseno, “Fast GPU-based interpolation for SAR backprojection,” *Progress In Electromagnetics Research*, vol. 133, pp. 259–283, 2013.
- [9] Y. Xu, Z. Zhang, L. Chen, and L. Yang, “The adaptive streaming SAR back-projection algorithm based on half-precision in GPU,” *Electronics*, vol. 11, no. 18, art. 2807, 2022.
- [10] O. Frey, C. Magnard, M. Rüegg, and E. Meier, “Focusing of airborne synthetic aperture radar data from highly nonlinear flight tracks,” *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 6, pp. 1844–1858, 2009.
- [11] O. Ponce, P. Prats, M. Rodriguez-Cassola, R. Scheiber, and A. Reigber, “Processing of circular SAR trajectories with fast factorized back-projection,” in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, 2011, pp. 3692–3695.

- [12] A. W. Doerry, “Wavefront curvature limitations and compensation to polar format processing for synthetic aperture radar images,” Sandia National Laboratories, Albuquerque, NM, Rep. SAND2007-0046, 2007.
- [13] L. A. Gorham and B. D. Rigling, “Scene size limits for polar format algorithm,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. 52, no. 1, pp. 73–84, 2016.
- [14] C. V. Jakowatz, Jr. and N. Doren, “Comparison of polar formatting and back-projection algorithms for spotlight-mode SAR image formation,” in *Proc. SPIE 6237, Algorithms for Synthetic Aperture Radar Imagery XIII*, 2006, art. 62370H.
- [15] N. P. Jouppi *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *Proc. 44th Annu. Int. Symp. Computer Architecture (ISCA)*, Toronto, ON, 2017, pp. 1–12.
- [16] NVIDIA Corporation, “CUDA C++ Programming Guide,” Section 5.4.1, Arithmetic instructions, 2025. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [17] Google Cloud, “Introducing Trillium, sixth-generation TPUs,” May 2024. [Online]. Available: <https://cloud.google.com/blog/products/compute/introducing-trillium-6th-gen-tpus>
- [18] Introl, “Google TPU architecture: 7 generations explained,” 2025. [Online]. Available: <https://introl.com/blog/google-tpu-architecture-complete-guide-7-generations>
- [19] GPU Advisor, “TPU v6e (Trillium) specs,” 2025. [Online]. Available: <https://gpuadvisor.com/gpu/tpu-v6e>
- [20] N. P. Jouppi *et al.*, “TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings,” in *Proc. 50th Annu. Int. Symp. Computer Architecture (ISCA)*, Orlando, FL, 2023.
- [21] T. Lu, Y.-F. Chen, B. Hechtman, T. Wang, and J. Anderson, “Large-scale discrete Fourier transform on TPUs,” arXiv:2002.03260, 2020.
- [22] T. Lu, T. Marin, Y. Zhuo, Y.-F. Chen, and C. Ma, “Nonuniform fast Fourier transform on TPUs,” in *Proc. IEEE 18th Int. Symp. Biomedical Imaging (ISBI)*, 2021.
- [23] A. G. M. Lewis *et al.*, “Large-scale distributed linear algebra with tensor processing units,” *Proc. Natl. Acad. Sci. USA*, vol. 119, no. 33, 2022.
- [24] D. Macdonald, “RITSAR: Synthetic aperture radar simulation and processing toolbox for Python,” 2015. [Online]. Available: <https://github.com/dm6718/RITSAR>
- [25] California Institute of Technology, “ISCE3: InSAR Scientific Computing Environment, version 3,” release 0.25.17. [Online]. Available: <https://github.com/isce-framework/isce3>
- [26] H. Forstén, “torchbp: Fast SAR backprojection with PyTorch,” 2025, commit b371884. [Online]. Available: <https://github.com/Tt1/torchbp>
- [27] L. A. Gorham and L. J. Moore, “SAR image formation toolbox for MATLAB,” in *Proc. SPIE 7699, Algorithms for Synthetic Aperture Radar Imagery XVII*, 2010, p. 769906.

- [28] J. W. McCorkle and M. Rofheart, “An order $N^2 \log(N)$ backprojector algorithm for focusing wide-angle wide-bandwidth arbitrary-motion synthetic aperture radar,” in *Proc. SPIE 2747, Radar Sensor Technology*, 1996, pp. 25–36.
- [29] S. Xiao, D. C. Munson, Jr., S. Basu, and Y. Bresler, “An $N^2 \log N$ back-projection algorithm for SAR image formation,” in *Conf. Rec. 34th Asilomar Conf. Signals, Systems and Computers*, 2000, pp. 3–7.
- [30] L. M. H. Ulander, H. Hellsten, and G. Stenström, “Synthetic-aperture radar processing using fast factorized back-projection,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. 39, no. 3, pp. 760–776, 2003.
- [31] W. G. Carrara, R. S. Goodman, and R. M. Majewski, *Spotlight Synthetic Aperture Radar: Signal Processing Algorithms*. Norwood, MA: Artech House, 1995.
- [32] C. V. Jakowatz, Jr., D. E. Wahl, P. H. Eichel, D. C. Ghiglia, and P. A. Thompson, *Spotlight-Mode Synthetic Aperture Radar: A Signal Processing Approach*. Boston, MA: Kluwer, 1996.
- [33] L. R. Rabiner, R. W. Schafer, and C. M. Rader, “The chirp z-transform algorithm,” *IEEE Trans. Audio Electroacoust.*, vol. 17, no. 2, pp. 86–92, 1969.
- [34] T. T. Taylor, “Design of line-source antennas for narrow beamwidth and low side lobes,” *IRE Trans. Antennas Propag.*, vol. 3, no. 1, pp. 16–28, 1955.
- [35] J. F. Kaiser, “Nonrecursive digital filter design using the I_0 -sinh window function,” in *Proc. IEEE Int. Symp. Circuits and Systems*, 1974, pp. 20–23.
- [36] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: SIAM, 2002.
- [37] S. Wang and P. Kanwar, “BFloat16: The secret to high performance on Cloud TPUs,” Google Cloud Blog, Aug. 2019.
- [38] J. Bradbury *et al.*, “JAX: Composable transformations of Python+NumPy programs,” 2018. [Online]. Available: <https://github.com/google/jax>
- [39] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in *Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC*, 2015.
- [40] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, “CuPy: A NumPy-compatible library for NVIDIA GPU calculations,” in *Proc. Workshop on Machine Learning Systems (LearningSys) at NIPS*, 2017.
- [41] A. H. Barnett, J. Magland, and L. af Klinteberg, “A parallel nonuniform fast Fourier transform library based on an ‘exponential of semicircle’ kernel,” *SIAM J. Sci. Comput.*, vol. 41, no. 5, pp. C479–C504, 2019.

A Algorithms

The phase history is motion compensated to the scene reference point. For pulse p with antenna position $\mathbf{a}_p$ and frequency sample f_k ,

$$S_{p,k} = \sum_n \sigma_n \exp\left(-j \frac{4\pi f_k}{c} \Delta R_p(\mathbf{x}_n)\right), \quad \Delta R_p(\mathbf{x}) = \|\mathbf{x} - \mathbf{a}_p\| - \|\mathbf{a}_p\|, \quad (1)$$

where σ_n and $\mathbf{x}_n$ are the complex amplitude and position of scatterer n and c is the speed of light. The CPHD files give the transmit and receive positions per pulse, whose midpoint was used as $\mathbf{a}_p$, and the start frequency and sample spacing per pulse; the mean start frequency and spacing were used, which differ from the per-pulse values by at most 0.02 of a sample spacing. A Taylor weighting ($\bar{n} = 4$, -35 dB sidelobes) [34] was applied along both axes in every algorithm. The image grid is the vendor's: pixel (i, j) lies at $(i - n_x/2) s_x \mathbf{e}_1 + (j - n_y/2) s_y \mathbf{e}_2$ from the reference point, with $\mathbf{e}_1$ and $\mathbf{e}_2$ the SICD column and row unit vectors and s_x, s_y its sample spacings.

Exact backprojection. Each pulse is range compressed by a zero-padded inverse FFT, with 8 times oversampling in the selected configurations and 16 times in the reference and in the two rows so marked, and each pixel is the sum over pulses of the linearly interpolated profile at ΔR_p times the carrier phase $\exp(j4\pi f_c \Delta R_p/c)$ [27]. The range difference is evaluated in the cancellation-free form

$$\Delta R_p(\mathbf{x}) = \frac{q}{1 + \sqrt{1 + q/r_p}}, \quad q = \frac{\|\mathbf{x}\|^2}{r_p} - 2 \hat{\mathbf{u}}_p \cdot \mathbf{x}, \quad (2)$$

with $r_p = \|\mathbf{a}_p\|$ and $\hat{\mathbf{u}}_p = \mathbf{a}_p/r_p$. The reference implementation evaluates everything in float64 with Numba [39] on the CPU. The CUDA kernel, compiled through CuPy [40], evaluates the base range, the unit vector and the reciprocal range of each 32 by 32 pixel tile in float64 once per pulse, and the per-pixel remainder relative to the tile center in float32; the pulses are processed in chunks of 256 so that the range profiles of one chunk fit in memory at any image size. In the first round the per-pixel geometry was computed in float32 from the scene center and its error grew with the size of the scene; computed from the tile center the error does not grow.

Factorized backprojection. The image is divided recursively into tiles, as in the quadtree [28, 29] and fast factorized [30] backprojection algorithms. At each level the phase history of a parent tile is multiplied by $\exp\{j4\pi f_k(\|\mathbf{c} - \mathbf{a}_p\| - \|\mathbf{c}' - \mathbf{a}_p\|)/c\}$ to move its reference point from the parent center $\mathbf{c}'$ to a child center $\mathbf{c}$; the child's signal is then band limited in frequency and in pulse index in proportion to the child's size, and both axes are low-pass filtered and decimated by a Kaiser-windowed sinc [35] with 70 dB design attenuation. At the last level the range to each pixel is linearized about the tile center, the kernel separates in the two image coordinates, and the tile image is a product of two matrices followed by a per-pixel quadratic phase correction, so that no stage requires an indexed read. The implementation handles rectangular images of any size: the side of each level divides the padded image by factors chosen from 8, 7, 6, 5, 4, 3 and 2, with 32 by 32 pixel final tiles, and the decimation factors follow from the range and Doppler extents of a child tile. For the Panama image the three levels divide the azimuth side by 8, 8 and 6 and the range side by 8, 7 and 5, and the first level decimates the 14,399 samples by 6 and the 15,186 pulses by 4. The first two levels compute their phase ramps from float64 geometry on the host and the remaining level from float32 geometry on the device; in the first round a float32 second level was found to add an error that grew with the size of the scene. The ramps themselves are evaluated on the device in float32, either directly (*direct ramps* in the tables) or as the product of a coarse and a

Table 5: Floating-point type of each step in the timed configurations. Geometry means tile centers, ranges and phase-ramp coefficients; ramps means the sines and cosines themselves; products means the operands of the matrix products or convolutions, always accumulated in float32 or better.

Device	Algorithm, label	Data in memory	Geometry	Ramps	Products
L4	Exact BP, float32 profiles	float32 profiles	float64 per tile	float32	float32 sums
L4	Exact BP, float16 profiles	float16 profiles	float64 per tile	float32	float32 sums
L4	Factorized, float32, CUDA kernels	float32	float64	float32	float32 fused multiply-adds (levels and final)
L4	Factorized, float16 throughout, CUDA kernels	float16	float64	float32	float16 data into float32 fused multiply-adds in the levels; float16 operands on the tensor cores in the final stage
L4	Factorized, float16 products	float16	float64 (2 levels), float32	float32	float16 operands, tensor cores
L4	Factorized, float32	float32	float64 (2 levels), float32	float32	float32 (highest setting)
L4	Factorized, TF32 products	float32	float64 (2 levels), float32	float32	TF32 operands (default setting)
TPU	Factorized, single-pass, fused kernels	float32	float64 (2 levels), float32	float32	bfloat16 operands, 1 pass (levels and final)
TPU	Factorized, three-pass, fused kernels	float32	float64 (2 levels), float32	float32	3 bfloat16 passes (levels and final)
TPU	Factorized, single-pass	float32	float64 (2 levels), float32	float32	bfloat16 operands, 1 pass
TPU	Factorized, three-pass	float32	float64 (2 levels), float32	float32	3 bfloat16 passes
TPU	Factorized, six-pass	float32	float64 (2 levels), float32	float32	6 bfloat16 passes
TPU	Factorized, bfloat16 storage	bfloat16	float64 (2 levels), float32	float32	bfloat16 operands, 1 pass
TPU	Polar format, float32	float32	float64	float32	resampling product 6 passes; float32 FFTs
TPU	Polar format, single-pass product	float32	float64	float32	resampling product 1 pass; float32 FFTs
L4, CPU	Polar format, float32	float32	float64	float32	float32 products and FFTs
CPU	Factorized, float32, C++ kernels	float32	float64	float32	float32 fused multiply-adds (levels and final)
CPU	Factorized, float32	float32	float64 (2 levels), float32	float32	float32
CPU	Factorized, float64	float64	float64	float64	float64

fine table (the *split-table* form, about $2\sqrt{n}$ sines and cosines per ramp of n samples, used by every row not labeled *direct ramps*); the device times of both forms are in Appendix J. The decimation filters are applied as dense matrix products (the TPU form), as strided convolutions (the L4 and CPU form) or as explicit sums over taps; on the TPU the three give the same image to better than -140 dB, and on the L4 the convolution differs from the dense product by -64 to -76 dB with TF32 products and not at all with full float32 products. One first-level tile is processed per call to the device, which bounds the working memory to about 6 GB at these sizes on every device. Polar format is described in Appendix E.

B Floating-point formats and the arithmetic of each configuration

Table 5 states, for every configuration that was timed, in which floating-point type each step runs. A first-round configuration in which the phase ramps were also computed in bfloat16 produced an unusable image and is excluded. The L4’s float16-product configuration also holds the data in float16; on the TPU the equivalent change, bfloat16 storage, is timed separately. The gather form of polar format is the selected form on the L4, the CPU and the v5e, and on the v5e the only form that fits, because the dense 15,186 by 15,186 resampling matrix and its intermediates exceed that device’s 16 GB; it has no resampling product, and its arithmetic is that of the L4 and CPU row.

Table 6 lists the formats that appear in this report. Float16 (IEEE binary16) keeps 11 significant bits and a 5-bit exponent, so it resolves 1 part in 2048 but overflows above 65,504; the phase history is therefore scaled to unit peak before conversion. Bfloat16 [37] keeps the 8-bit exponent of float32 and only 8 significant bits, 1 part in 256. On the TPU every matrix product is formed from bfloat16 operands. A float32 operand is either rounded once (one pass), or split into a bfloat16 head and tail with the cross terms accumulated (three passes), or split into three parts (six passes), which recovers float32 accuracy [38]. The L4’s tensor cores accept float16 and bfloat16 operands, and TF32 (11 significant bits) for float32 operands at JAX’s default precision (Section 4). The CPU has no reduced-precision arithmetic that is faster than float32.

The measured image errors agree with the operand resolutions of Table 6. Rounding each operand of the factorized products to bfloat16 adds noise about 48 dB below the operand [36], and the measured error of the single-pass TPU image, once the algorithm’s own error (the float64 row of Table 13) is removed, matches the rounding noise of the two operands accumulated through three levels. The L4’s float16 operands have 18 dB finer resolution than bfloat16, and their rounding adds 0.9 dB to the error of the float32 CUDA build, which multiplies in float32 (1.1 dB in the JAX

Table 6: Floating-point formats. Significant bits include the implicit bit; the spacing is the distance between adjacent representable values near 1.

Format	Bits	Significant bits	Exponent bits	Spacing near 1
float64	64	53	11	2.2×10^{-16}
float32	32	24	8	1.2×10^{-7}
TF32 (GPU product operand)	19	11	8	9.8×10^{-4}
float16 (IEEE binary16)	16	11	5	9.8×10^{-4}
bfloat16	16	8	8	7.8×10^{-3}

program, where the float16 configuration also multiplies in float16). The three-pass product keeps the first 16 significant bits of each operand.

C Using the TPU

Survey. In the first round the gather that backprojection needs per pixel and pulse was measured on the TPU at about 10^8 reads per second; the L4’s CUDA kernel performs the 1.6 trillion reads of the Panama image in 104 s, about 1.5×10^{10} per second. The public documentation of the TPU generations, the JAX and Pallas documentation and the published work on irregular computation on TPUs were searched in October 2026 for a way to run exact backprojection, or any lookup-based formulation, at a rate comparable with the L4. A TPU core has a matrix unit, which multiplies dense tiles of bfloat16 operands at the chip’s rated rate, and a vector unit, which executes element-wise operations and gathers at a rate set by memory bandwidth and lane count, two orders of magnitude lower per element. No texture or cache hierarchy for scattered reads is documented. The v6e and TPU7x add sparse cores for embedding lookups, which Google’s Cloud TPU documentation (accessed October 2026) places near 10^9 rows per second per chip and which are programmed through Pallas, JAX’s kernel language; at that rate the 1.6 trillion reads of the Panama image would take about 30 minutes, against 100 to 200 seconds on the L4. Writing the lookup as a one-hot matrix product, so that the matrix unit does it, multiplies by a matrix whose density is one in several thousand, and the arithmetic cost exceeds the dense algorithm’s by the same factor. No published work was found that forms SAR, beamforming or tomographic images on a TPU; the nearest are formulations of FFTs and non-uniform FFTs as matrix products [41], the approach taken here. The documentation’s recommendations for dense workloads were applied and tested: shapes padded to multiples of 128 (v5e) or 256 (v6e), bfloat16 operands for default products, explicit precision per product and profiling of the compiled program; the recommendation for the TPU7x (two devices per chip) could not be tested.

Profile. Figure 5 shows the time of each stage of the first level on one first-level tile of the Panama collection, 15,186 pulses by 14,399 samples decimated to 3,805 by 2,408, measured one stage at a time on the v6e with single-pass products and on the L4 with TF32 products (profiling one stage at a time materializes the inputs and outputs of each stage, which the fused program does not; for a whole tile these exceed the v5e’s 16 GB, so the v5e was not profiled). The phase ramp fused with the complex multiplication (rotate) costs 9.8 ms on the v6e; the frequency-axis decimation as a dense product 3.5 ms, as a strided convolution 16.4 ms and as a block-banded product 10.9 ms; the pulse-axis decimation 1.0, 12.2 and 3.0 ms. The dense products run at 584 TFLOP/s including the multiplications by the zeros of the filter matrices, and the rotation moves 534 GB/s through

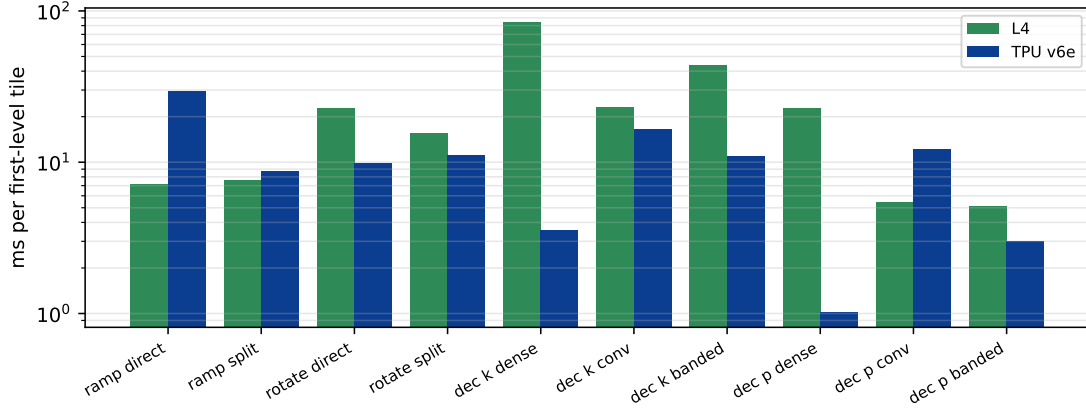


Figure 5: Time per first-level tile of each stage of the factorized algorithm, by form, on the TPU v6e (single-pass products) and the L4 (TF32 products), with float32 data. Logarithmic scale.

memory. On the L4 the same stages take 22.7 ms for the rotation and 83.9, 23.1 and 43.6 ms for the three forms of the frequency-axis decimation, so the strided convolution was used on the L4 and the dense product on the TPU. The deeper levels of the factorization multiply the number of children faster than the decimation shrinks the data, so the element-wise rotation accounts for most of the TPU device time over the whole image; the matrix units are active for roughly a third of the first level and for less of the rest.

Measures that did not reduce the JAX program’s TPU time. (The kernels of Appendix D supersede this program on the TPUs; the figures here describe it.) Padding the pulse and sample counts to multiples of 256 increased the v6e device time from 8.3 to 8.9 s (both timed in one session, earlier than the 8.0 s of the supplementary tables); the padded samples are carried through every level. Bfloat16 storage slowed both TPUs on the Panama and Melbourne images (5.2 compared with 4.5 s on the v6e), because the ramps remain float32 and the resulting conversions take longer than the reduction in memory traffic recovers; on the smaller Iowa image the difference was about 10% in either direction. The split-table ramps, which cut the number of transcendental evaluations by a factor of about 60, were slower than direct evaluation on both TPUs (4.5 compared with 3.8 s on the v6e). The compiler fuses the direct form into the multiplication, whereas the table products require a pass over memory of their own. At the stage level the strided convolution was five times slower than the dense product on the frequency axis and twelve times on the pulse axis (Figure 5); at the image level the factor was about four for Panama and Melbourne and two to three for Iowa on both TPUs (Appendix J).

Host transfers. A 1.75 GB collection moves from host memory to the v6e at 14 GB/s as two float32 planes, and an order of magnitude more slowly as a complex64 array, which passes through a conversion; the same holds for the image on the return path. The pipelined measurements move planes in both directions; before this change, the polar-format throughput on the v6e corresponded to 3.0 s per image against a device time of 1.3 s.

D Kernels for the factorized algorithm

The factorized rows of Table 3 are formed by kernels written for each device after a stage-by-stage profile of the JAX program, that is, of the code XLA (JAX’s compiler) generates from it; this appendix records the diagnosis, the kernels, the time each step removed (Table 8) and the bounds the final builds reach (Table 7). Run as a single JAX program, the factorized algorithm took 4.6, 9.3, 13.7 and 217.4 s for the float32-accuracy Panama image on the v6e, v5e, L4 and CPU, and its cost per thousand images, from the pipelined loops, was within 11% across the three accelerators; the kernels made the TPUs 1.9 to 2.0 times faster at that accuracy, the L4 3.8 times and the CPU 17.6 times, and the float32 images of the CUDA and C++ kernels are closer to the reference than those of the JAX program because they evaluate the geometry of every level in float64. In the terms of Appendix A, a level splits each parent tile’s phase history into those of its children, the tiles of the next level. Level 0 splits the whole collection into the first-level tiles; levels 1 and 2, which run inside one first-level tile per device call, are called the tile levels below. Of these, only level 2 takes its geometry from the device. The plan, filters, geometry and arithmetic order are the same in every build, and each build’s image was checked against the dense image at the same precision on a simulated scene before it was timed: single-pass and float16 builds agree with the dense image to the level of their own rounding, three-pass and float32 builds to below -80 dB.

Diagnosis. On the v6e, one first-level tile of the Panama image took 51 ms through the XLA program with single-pass products: 8.3 ms in the first level, 13.3 in the second, 7.9 in the third and 21.4 in the final stage. The three levels are bound by memory traffic: XLA writes the rotated phase history of every child (24 bytes per sample, read back by the decimation product) at about 580 GB/s, a third of the v6e’s 1,640 GB/s, and the children of the three levels total 63 billion samples per image. The final stage materializes the two ramp matrices of every 32 by 32 tile, $2T$ by $P_f Q_f$ each, through five or six element-wise passes before the one product that uses them, and its direct form evaluates 400,000 sines and cosines per tile. The same structure holds on the L4 at the convolution path’s lower bandwidth.

TPU kernels (Pallas). Two kernels, written in Pallas (JAX’s kernel language for the TPU), replace the XLA stages. The level kernel takes a block of pulses and a window of columns of a parent into the core’s vector memory and applies the phase ramps of up to eight children in turn. Each ramp is the product of a coarse table, one angle per 128-column chunk, and a fine table, one angle per column within the chunk; both are precomputed once per pulse and child by XLA, so the kernel evaluates no sines (Pallas lists them as its most expensive operations, and an earlier build that evaluated the coarse table in the kernel spent a third of its time on them). The rotated window is multiplied by the banded block of the decimation matrix on the matrix units, in one or three bfloat16 passes, and at the two device levels the pulse-axis decimation is accumulated on the matrix units across the pulse blocks in the same kernel, so neither the rotated window nor the frequency-decimated child reaches high-bandwidth memory; at level 0 the decimated child is too large for vector memory and the pulse product stays a separate dense product. Children of one parent share each load, the device levels batch the parents of a tile into one launch, and first-level tiles are formed in groups of sixteen so that the phase history itself is read once per group. Where one block of outputs covers a whole row (the device levels), the window is clipped to the data and read once; at level 0 consecutive output blocks overlap by the filter length.

The final kernel forms the $2T \times 2T$ product of Appendix A ($T = 32$ pixels on a side, $P_f \times Q_f$ pulses by frequency samples per tile) on the matrix units, with both ramps generated by a complex recurrence along the frequency index q : for a pixel offset d along an image axis and the component

u_p of the unit vector to the antenna along that axis, the ramp $\exp(-j2\pi du_p(a_0 + a_1q))$, with a_0 and a_1 the cycles per meter at the first frequency sample and per sample step, is XW^q with $X = \exp(-j2\pi du_p a_0)$ and $W = \exp(-j2\pi du_p a_1)$, so one tile needs four tables of sines and cosines of size $T \times P_f$ instead of one sine per sample. A first version of this kernel with one sine per sample was no faster than XLA (Table 8), which placed that stage on the transcendental unit; the recurrence version removed it. Three variants were measured and rejected (Table 8): building the level kernel’s fine tables by the same doubling idea in XLA doubled the image time, because each intermediate table of fewer than 128 columns occupies full 128-lane tiles in TPU memory; evaluating the coarse-table sines over the used lanes only and padding afterwards cost 4 to 8%, because XLA holds the table 128 lanes wide either way and the pad is a copy; and stacking four tiles into one 256 by 256 product in the final stage, to fill the systolic array, gained 3% on the v6e at three-pass precision and nothing otherwise, and lost 2 to 4% on the v5e, whose arrays are 128 by 128.

Bounds. Table 7 compares each stage with the bound set by the unit that binds it. The unit rates were measured on each device with microbenchmarks in the same framework (`ubench_tpu.py`, `ubench_gpu.py` in the repository): on the v6e, high-bandwidth memory copies at 1,391 GB/s; the vector unit executes the level kernel’s own rotation mix on data resident in vector memory at 117 billion samples per second and a plain copy through vector memory at 155 billion, so the rotation runs within 1.3 times of the vector load and store path itself; XLA evaluates 57 billion sine and cosine pairs per second; the matrix units reach 457 TFLOP/s on a large bfloat16 product and 42 at the final stage’s 64-row shape. Against these, on the v6e, level 1 runs at the vector unit’s rate (ratio 1.0), level 0 at 1.4 times its vector bound with the unfused pulse product passing through memory, and level 2 at 2.5 times its largest bound but at 0.9 of the sum of the terms that run in sequence: the rotation (0.24 s per image), the sines XLA evaluates for its tables (0.25 s; the level’s rows are 352 samples long, so each child and pulse needs 256 sines for 512 rotated samples) and the traffic of those tables (0.17 s). The final stage is 2.2 times its matrix bound; the remainder is the ramp recurrence on the vector unit, which the microbenchmarks did not isolate. On the v5e the matrix units, about three times slower than the v6e’s on a large product against a vector unit only 12% slower, become the binding unit of levels 0 and 1, and every stage sits between its largest bound and the sum of its bounds (1.5 to 2.2 times the former, 0.6 to 0.8 of the latter), that is, its units overlap partly. The one lever this leaves inside the factorization is the level-2 table generation, which angle-addition tables could cut by about eight times for roughly 0.2 s of the v6e image’s 1.7; every other stage is at most 2.2 times its largest single bound and below the sum of its bounds.

L4 kernels (CUDA). On the GPU the natural form of the same two kernels is CUDA, compiled through CuPy. The level kernel loads the window of 32 pulses into shared memory with consecutive threads on consecutive columns, which is the access pattern the memory system serves in full-width transactions, and rotates it in place for each child, the rotation of each thread’s samples generated as a geometric sequence. Each thread then computes two adjacent outputs of one pulse with the 44-tap decimation filter as fused multiply-adds; for this phase consecutive threads are consecutive pulses, so no two threads of a group read the same shared-memory bank, and the outputs are staged through shared memory and written row by row. The pulse-axis filter is tiled the same way. The final kernel builds the two ramp tables of a tile in shared memory by the same recurrence and each thread accumulates a 4 by 2 block of pixels; in the sixteen-bit build the tables are converted to float16 and the $2T \times 2T$ product runs on the tensor cores with float32 accumulation. A Triton version of the level kernel through Pallas, with the 16-column windows its shared-memory limit allows, gained one to four percent at sixteen bits and was slower in float32; the CUDA builds are

what Table 3 reports.

The CUDA code went through seven builds, five of them profiled (Table 8; the profiles are in the repository). The profile of the second build placed 1.5 s per image in the level kernel, 1.3 in the final stage and 0.5 in the pulse filter. The pulse-major thread mapping of the third build freed the filter reads of bank conflicts but made the global loads and stores of that kernel scattered, and its time doubled; coalesced loads through shared memory, the geometric-sequence rotation and the staged stores brought it to 1.9, 1.7 and 1.4 s in the fourth, fifth and sixth builds. The memory access pattern changed the time by more than the sine count did. The float32 final stage was bound by the latency of its table-building recurrence and by one shared-memory word per fused multiply-add; the 4 by 2 block per thread with interleaved complex tables brought it from 1.3 to 1.0 s. The tensor-core version is 0.4 s faster per image than the float32 one in the whole-image measurement and 0.2 dB less accurate. The seventh build stores the phase history, every intermediate and the final-stage operands in float16 and accumulates in float32 inside the filters. The float16 storage halves the memory traffic that bounds the level kernel and the pulse filter. It forms the image in 2.9 s against 3.6 s for the float32 build, with 0.9 dB more error, and is the float16 configuration of Table 3. The sixth build with the tensor-core final stage and float32 storage is the intermediate setting at 3.2 s.

Against the L4’s measured rates (memory copy 246 GB/s; the level kernel’s inner mix on resident shared memory 20.2 billion filter outputs per second; the final kernel’s inner mix 6.1 trillion fused multiply-adds per second, 41% of the chip’s peak; coalesced stores 252 GB/s against 69 strided), the pulse filter runs at its memory bound, the level kernel at 1.1 times the sum of its arithmetic and memory bounds (1.9 times their maximum; the two overlap only partly at two blocks per multiprocessor), and the float32 final stage at 2.2 times the bound of its own inner mix, the difference being the table-building phase. The remaining structural cost of the float32 build is the 172 GB per image that the level kernel writes and the pulse filter reads back; a streaming fusion of the two could remove about half of it, worth about 0.3 s of the 3.6. The float16 build halves that traffic instead and gains 0.7 s on the float32 time, which is consistent with the level kernel’s time being mostly its arithmetic bound. Float16 storage does not move that bound, because the L4’s CUDA cores issue float16 and float32 fused multiply-adds at the same rate (128 per clock per multiprocessor at compute capability 8.9 [16]). The remaining float16 route on this card is the tensor cores, which would require rewriting the level filters as matrix products in the manner of the TPU kernel.

CPU kernels (C++). The level filter, the pulse filter and the final stage were written in C++ with OpenMP for the 8-core instance, compiled by GCC 13 for AVX-512 with 512-bit vectors preferred and called from the Python driver through a shared library; the host side (plan, filters, float64 geometry) is the code used for the other devices. The design that approached the arithmetic bound puts the children of a parent in the vector lanes: a sample of the parent row is a scalar broadcast, its rotation one complex multiply per 16 children with a per-lane phase step, and the filter accumulates eight outputs per child vector in registers, so no strided read occurs. The pulse filter reads each input row once per chunk of 16 output rows, and the final stage builds its two tables three pulses at a time, by a geometric recurrence over the pulse index and another over the pixel index (four sines per pulse and table), so that they stay in the second-level cache while the product consumes them. Four builds were measured (Table 8). The first, a polyphase filter vectorized along the output columns, took 51.8 s per Panama image, because its stride- D gathers cost about twelve instructions per useful multiply-add. Assigning children to vector lanes brought the image to 19.8 s (second build). Reusing the intermediate buffers, which removed one page fault per 4 KB of fresh allocation inside the parallel regions, and keeping the accumulators in registers brought it to 13.2 s (third build). Building and consuming the final-stage tables three pulses at a time brought the

Table 7: Each stage of the final kernel builds against the lower bound set by the unit that binds it, Panama image, single-pass or float32 arithmetic. The bounds divide the inherent work of the stage by rates measured on the same device with microbenchmarks in the same framework (vector unit on the rotation mix of the kernel with data resident on chip, high-bandwidth memory copy, matrix units at the product shapes of the kernel, sines in XLA; on the L4 the inner mixes of the kernels on resident shared memory and the memory copy rate). A ratio near one means the stage runs at the rate of that unit; the last column divides the measured time by the sum of all the bounds of the stage, the time it would take if none of its units overlapped.

Device	Stage	Binding unit	Bound (s)	Measured (s)	Ratio	To the sum
TPU v6e	level 0	vector unit	0.15	0.22	1.4	0.60
	level 1	vector unit	0.30	0.30	1.0	0.52
	level 2	sines (XLA)	0.25	0.61	2.5	0.88
	final	matrix units	0.13	0.29	2.2	–
TPU v5e	level 0	matrix units	0.33	0.50	1.5	0.72
	level 1	matrix units	0.38	0.60	1.6	0.62
	level 2	sines (XLA)	0.40	0.88	2.2	0.78
	final	matrix units	0.25	0.44	1.7	–
L4	levels (rotation and frequency filter)	memory	0.73	1.36	1.9	1.08
	pulse filter	memory	0.41	0.46	1.1	1.11
	final (float32)	arithmetic	0.45	0.98	2.2	2.07
CPU	levels (rotation and frequency filter)	arithmetic	4.19	4.87	1.2	0.79
	pulse filter	memory	2.06	2.17	1.1	0.84
	final (float32)	arithmetic	2.67	3.76	1.4	1.35

image to 12.4 s, with the final stage at 3.8 s (fourth build). The fourth build, used in Table 3, forms the Panama image in 12.4 s on 8 threads, one per core (16 threads gave the same time), 17.6 times faster than the JAX program. Against the instance’s measured rates (memory copy 49 GB/s; the filter’s inner loop 314 billion fused multiply-adds per second on resident data; the final product’s inner loop 1.04 trillion fused multiply-adds per second, near the 8 cores’ float32 peak of two 512-bit fused multiply-adds per core and cycle), the level kernel runs at 1.2 times its arithmetic bound (0.8 of the sum of arithmetic and memory), the pulse filter at 1.05 times its memory bound and the final stage at 1.4 times its arithmetic bound, the difference being the table building (Table 7).

Table 8: Device time per Panama image (s) of each build of the factorized algorithm, two precisions per accelerator and float32 on the CPU: sixteen-bit means single-pass products on the TPUs and, on the L4, float16 products (JAX program), the float32 build with the float16 tensor-core final stage (CUDA builds four to six) or float16 storage throughout (build seven); float32 class means three-pass products on the TPUs and float32 on the L4. Each row adds to the one above; the builds of record in Table 3 are marked; the float32-class entry in the last L4 row is a rerun of the float32 configuration of the sixth build.

Device	Build	16-bit (s)	float32 class (s)
TPU v6e	XLA: ramps materialized, dense decimation products, XLA final stage	3.82	4.60
	Fused level-0 kernel, one child per load	4.51	5.25
	Fused level kernels, children share a load, XLA final stage	4.11	4.75
	Same, device level batched over parents	3.07	3.72
	Plus fused final stage, per-sample sines	3.03	3.70
	Plus fused final stage, ramps by recurrence	2.01	2.75
	Plus clipped windows, no halo at levels 1 and 2, precomputed fine tables	1.90	2.62
	Fine tables by doubling instead (rejected)	4.06	4.78
	Coarse tables precomputed too, pulse decimation fused into the kernel at levels 1 and 2	1.69	2.36
	Four tiles per step in the final stage instead (3% faster on the v6e at three-pass precision only; not kept)	1.69	2.29
	Coarse-table sines over the used lanes only, padded afterwards (rejected)	1.80	2.46
	Generation-3 level kernel with the one-tile recurrence final (final build)	1.70	2.36
TPU v5e	XLA: ramps materialized, dense decimation products, XLA final stage	6.49	9.29
	Fused level-0 kernel, one child per load	7.72	10.11
	Fused level kernels, children share a load, XLA final stage	6.78	8.56
	Same, device level batched over parents	5.03	6.93
	Plus fused final stage, per-sample sines	5.13	7.20
	Plus fused final stage, ramps by recurrence	3.41	5.58
	Plus clipped windows, no halo at levels 1 and 2, precomputed fine tables	3.18	5.09
	Fine tables by doubling instead (rejected)	6.86	8.76
	Coarse tables precomputed too, pulse decimation fused into the kernel at levels 1 and 2	2.75	4.57
	Four tiles per step in the final stage instead (3% faster on the v6e at three-pass precision only; not kept)	2.85	4.67
	Coarse-table sines over the used lanes only, padded afterwards (rejected)	2.98	4.79
	Generation-3 level kernel with the one-tile recurrence final (final build)	2.75	4.57
L4	XLA: ramps materialized, cuDNN convolutions, XLA final stage	11.16	13.69
	Triton level-0 kernel (Pallas), 16-column windows	10.73	18.72
	CUDA: shared-memory FIR levels, one pixel per thread final	–	5.62
	Final stage 2 by 2 pixels per thread	–	4.61
	Pulse-major filter threads (conflict-free reads, scattered loads and stores), tiled pulse filter, 4 by 2 final	–	5.45
	Coalesced window loads, children as in-place phase steps; float16 tensor-core final	3.75	4.39
	Rotation as a geometric sequence per thread; interleaved complex tables in the final	3.55	3.98
	Output tiles staged in shared memory for coalesced stores (float32 build of record)	3.16	3.59
	Float16 phase history, intermediates and final-stage operands, float32 accumulation in the filters (float16 build of record)	2.90	3.51
CPU	JAX program: conv. filters, direct ramps, one process	–	217.40
	C++ with OpenMP: polyphase filter vectorized along the output columns (stride-D gathers)	–	51.84
	Children in the vector lanes (no gathers), input rows read once in the pulse filter, final tables by two recurrences	–	19.82
	Named accumulators, complex products as FMA pairs, 512-bit vectors, reused intermediates	–	13.24
	Final-stage tables built and consumed three pulses at a time (second-level cache); final build	–	12.36

E Polar format on an orbital aperture

Polar format assumes a planar wavefront, and at these scene sizes the assumption is not good enough on its own. A scatterer at distance ρ from the scene center is focused about $\rho^2/2R$ from where it belongs in range, which at $R = 744$ km (mid-aperture) is 4 m at 2.5 km, some ten pixels. The azimuth displacement is larger and depends on the squint: 7.6 m against 4.3 m at the Panama corners. The error is almost purely geometric. The fit below puts the corner displacement at up to 18 pixels in azimuth and 12 in range, the measured positions of bright targets agree with it to 0.2 pixel, and the image is sharp everywhere. Its 5 by 5 coherence against the reference is only 0.4206, but that is misregistration, not defocus.

Before its final resampling the image forms in 1.3 to 4.7 s depending on the device, the floor of what polar format can cost; its features sit up to eighteen pixels from their true positions at that point, so every polar-format figure in this report is for the resampled image.

The final resampling forms the image at twice the pixel density without carriers, resamples it at the apparent positions with a six-tap Lanczos kernel and restores the carriers. It brings the error to

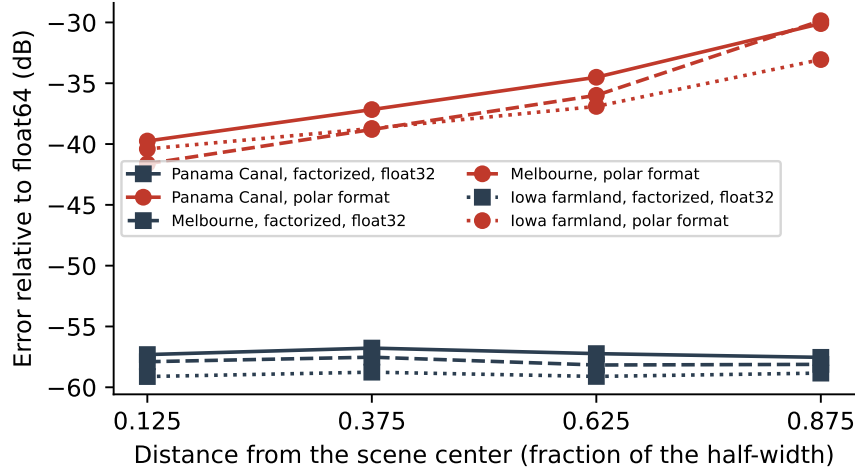


Figure 6: Error relative to the float64 reference in four concentric rings by distance from the scene center, for the polar-format image and the float32 factorized image on the L4, on the three collections.

−32.1 dB on Panama (−33.4 and −35.3 dB on the other collections), the coherence to 0.9994 and the registration to 0.001 pixel. The residual grows from −40 dB at the center to −30 dB in the outer quarter (Figure 6). Its cause was not identified. Varying the resampling kernel and the raster oversampling changed it by less than 0.6 dB, and longer pulse interpolators made it up to 0.9 dB worse. The registration error and the reference’s convergence are too small to account for it (one untested candidate is noted below).

The resampling is an indexed read at a computed position, 36 per pixel. On Panama it adds 0.8 s on the L4 and 6.7 s on the CPU, so the image takes 2.1 s on the L4 and 11.3 s on the CPU, 1.7 and 1.1 times faster than factorized backprojection on the same device. On the TPUs it takes 97.4 s (v5e) and 105.3 s (v6e), slower than factorized backprojection, because scattered gathers run on the TPU’s vector unit at about 4×10^7 reads per second. For production use on a TPU the resampling would have to move to the host or be replaced by an approximation without scattered gathers (Appendix H).

The implementation consists of three resampling steps and a two-dimensional FFT, each expressed with FFTs, element-wise products or a single matrix product so that the same code runs on all devices. The pulses are first resampled so that the tangent of the look angle in the image plane is uniform in pulse index, which an orbit does not give exactly; this is a 16-tap Kaiser-windowed sinc interpolation along the pulse axis applied as a dense 15,186 by 15,186 matrix (the form selected on the v6e) or as a sum over 16 gathered rows (the form selected on the L4, the CPU and the v5e; both forms were timed on the L4, the v6e and the CPU). Each pulse is then resampled onto a uniform radial wavenumber grid, and each wavenumber row onto a uniform cross-range grid, by chirp-z transforms [33], which evaluate the band-limited interpolation on a scaled grid with FFTs and chirps; outputs that would wrap around the band are zeroed. The frequency samples are weighted by f_c/f_k before resampling, because backprojection’s uniform weighting of the samples corresponds to a density of $1/k$ in the wavenumber plane (the polar Jacobian) that the uniform spectral weighting of polar format omits. Without this weighting the image’s error worsens by about 1.5 dB. It is applied in every reported polar-format image.

Because the grazing angle changes along the aperture, the radial band of each pulse projected

Table 9: Panama Canal, polar format on the CPU: whole-image error and error by ring (dB) for variations of the final resampling. Rings are the quarters of the half-width from the scene center outward, inner to outer; the selected configuration is the first row.

Variation	Error	Ring 1	Ring 2	Ring 3	Ring 4
6-tap kernel, 2 times oversampled (selected)	−32.1	−39.8	−37.2	−34.5	−30.1
4-tap kernel	−31.6	−37.1	−35.1	−33.5	−29.8
8-tap kernel	−32.2	−40.1	−37.4	−34.7	−30.2
3 times oversampled	−32.2	−40.1	−37.3	−34.7	−30.1
32-tap pulse interpolator, Kaiser 12	−31.7	−37.7	−35.7	−33.8	−29.8
48-tap pulse interpolator, Kaiser 12	−31.2	−36.4	−34.5	−33.1	−29.6

onto the image plane drifts from pulse to pulse. The raster covers the union of the bands and so keeps the full spectral support of backprojection, a trapezoid, rather than the inscribed rectangle; in the ground plane the rectangle would hold half the bandwidth, in the vendor’s slant plane nearly all of it. The wavenumber grid spacings are set by the native pixel spacings and FFT sizes of 13,824 by 10,800, which include a 300 m guard band on every side of the scene; the range and azimuth profiles are tapered to zero within the guard, because the periodic transforms otherwise wrap echoes from beyond the image into its edges (without the guard the edge error was −5 dB). The image is cut from the FFT output on the native grid with the carrier phases restored.

Distortion and the final resampling. The planar model writes the range history of a scatterer at $\mathbf{p}$ as $-\hat{\mathbf{u}}_p \cdot \mathbf{q} + c$; a least-squares fit of $\mathbf{q}$ and c over the aperture to the exact history $\|\mathbf{p} - \mathbf{a}_p\| - \|\mathbf{a}_p\|$ gives the position at which polar format focuses the scatterer, $(q_x, q_y + c)$, since a constant range offset is a range shift. The fit residual is below 1 mrad of phase everywhere in the three scenes, so before the final resampling the image is displaced without defocus, by up to 18 pixels in azimuth and 12 in range at the corners of the Panama scene; measured target positions agree with the model to 0.2 pixel. The mapping is fitted by cubic polynomials over the image (residual below 10^{-4} pixel).

The image is resampled through the mapping: the spectrum is placed in a raster of twice the pixel density, transformed without the carrier terms, interpolated at the apparent positions with a six-tap Lanczos kernel along each axis, and multiplied by the carriers evaluated at the apparent positions. No separate phase correction is needed; the quadratic phase of about 12 radians at 211 m seen in the center crop before the resampling is the carrier of the range displacement. The oversampled raster is processed in blocks rather than held in memory. The range transform is done once, and the azimuth transform and the resampling proceed in blocks of 512 output range columns.

The resampled image’s error against the reference is −39 dB in the central 1024 by 1024 pixels and −32 dB over the whole Panama image, growing toward the edges (the rings of Table 9 and Figure 6 are quarters of the half-width, not this crop). On the full Panama image the resampling kernel (4, 6 and 8 taps) and the oversampling of the intermediate raster (2 and 3 times) change the whole-image error by less than 0.6 dB and the outermost ring, where the displacement is 10 to 18 pixels, by less than 0.4 dB (Table 9); lengthening the pulse interpolator from 16 to 32 or 48 taps, with a narrower Kaiser window, made the error slightly worse. The registration, measured from the cross-spectrum phase, is 0.001 pixel, and the reference’s own convergence is more than 30 dB below the residual.

A remaining candidate, not tested, is the Taylor weighting, which backprojection applies per pulse and per sample but which polar format carries through the resampling onto the rectangular raster; the resulting difference would increase with distance from the center, as the residual does.

Table 10: The reference (float64 exact backprojection) against the vendor’s complex image (SICD) of the same collection, which Umbra forms by polar format. Columns: the number of 512 by 512 windows, centered on a grid with 1024-pixel spacing, with enough contrast for an amplitude cross-correlation; the largest displacement of the vendor image relative to the reference over the windows (azimuth / range, pixels); the largest displacement the planar-wavefront model of Appendix E predicts at the same windows; the root-mean-square difference between measured and predicted displacement; the root-mean-square residual displacement after the vendor image is resampled through the model; and, after that resampling and a local re-alignment and phase-ramp removal in each window, the 5 by 5 coherence of the two images (mean over all pixels of the scene; median of the window means) and the median over the windows of the correlation of their log amplitudes.

Collection	Windows	Measured (px)	Model (px)	RMS difference (px)	Residual after resampling (px)	Coherence, scene mean	window median	Log-amplitude correlation
Panama Canal	68	13.2 / 9.0	13.2 / 9.1	0.11 / 0.12	0.07 / 0.05	0.46	0.67	0.49
Melbourne	81	7.3 / 7.1	7.4 / 7.2	0.10 / 0.19	0.05 / 0.11	0.67	0.76	0.52
Iowa farmland	64	6.7 / 8.1	6.7 / 8.5	0.07 / 0.34	0.01 / 0.25	0.63	0.80	0.57

F Validation against the vendor’s image

Every error figure in this report is measured against the reference, the float64 exact backprojection of Section 5, implemented for this study. Umbra, the vendor, also distributes each collection as a complex image in the Sensor Independent Complex Data format (SICD), formed from the same phase history by its production processor using polar format, on the pixel grid that the reference adopts; this allows the reference to be checked against an independent processor. The integer and sub-pixel offset between the two images (which arises from the vendor’s reference-pixel convention) was found from the cross-correlation of block-averaged log amplitude and refined on a central window; the linear phase ramp between them, which reflects the different spectral centering of the two images (-0.225 cycles per pixel in range on Panama), was found from the peak of the cross-spectrum of the same window and removed. The vendor image was compared with the reference in 512 by 512 windows centered on a grid with 1024-pixel spacing, keeping the windows with enough contrast for an amplitude cross-correlation (68 on Panama). In each window the local offset of the vendor image was measured by amplitude cross-correlation and compared with the displacement that the planar-wavefront model of Appendix E predicts at the window center. The vendor image was then resampled through the model with a cubic kernel and, after a local re-alignment and phase-ramp removal in each window, the coherence and the log-amplitude correlation of the two images were computed. As delivered, the vendor image differs visibly from the reference in two respects: it is masked to the parallelogram the vendor’s processor considers fully supported, and bright points carry cross-shaped sidelobes that the reference’s Taylor weighting suppresses.

Table 10 gives the results. The vendor image is displaced from the reference by a field whose largest value over the Panama windows is 13.2 pixels in azimuth and 9.0 in range, against 13.2 and 9.1 predicted by the model from the collection geometry alone. The root-mean-square difference between measured and predicted displacement over the windows is 0.11 and 0.12 pixel on Panama and at most 0.34 pixel on the other two collections, 1 to 4% of the largest displacement. The vendor’s processor therefore places scatterers where the planar-wavefront model predicts that polar format places them, which supports both the displacement model and the geometry of the reference. After the resampling the root-mean-square residual displacement is at most 0.25 pixel (individual windows up to 0.43), so the reference is registered to the vendor’s grid to a small fraction of a pixel once the polar-format displacement is accounted for; the coherence of the two images averages 0.46, 0.67 and 0.63 on Panama, Melbourne and Iowa (0.67, 0.76 and 0.80 as the median over windows), and their log amplitudes correlate at 0.49 to 0.57 within windows (Figure 7). Closer agreement is

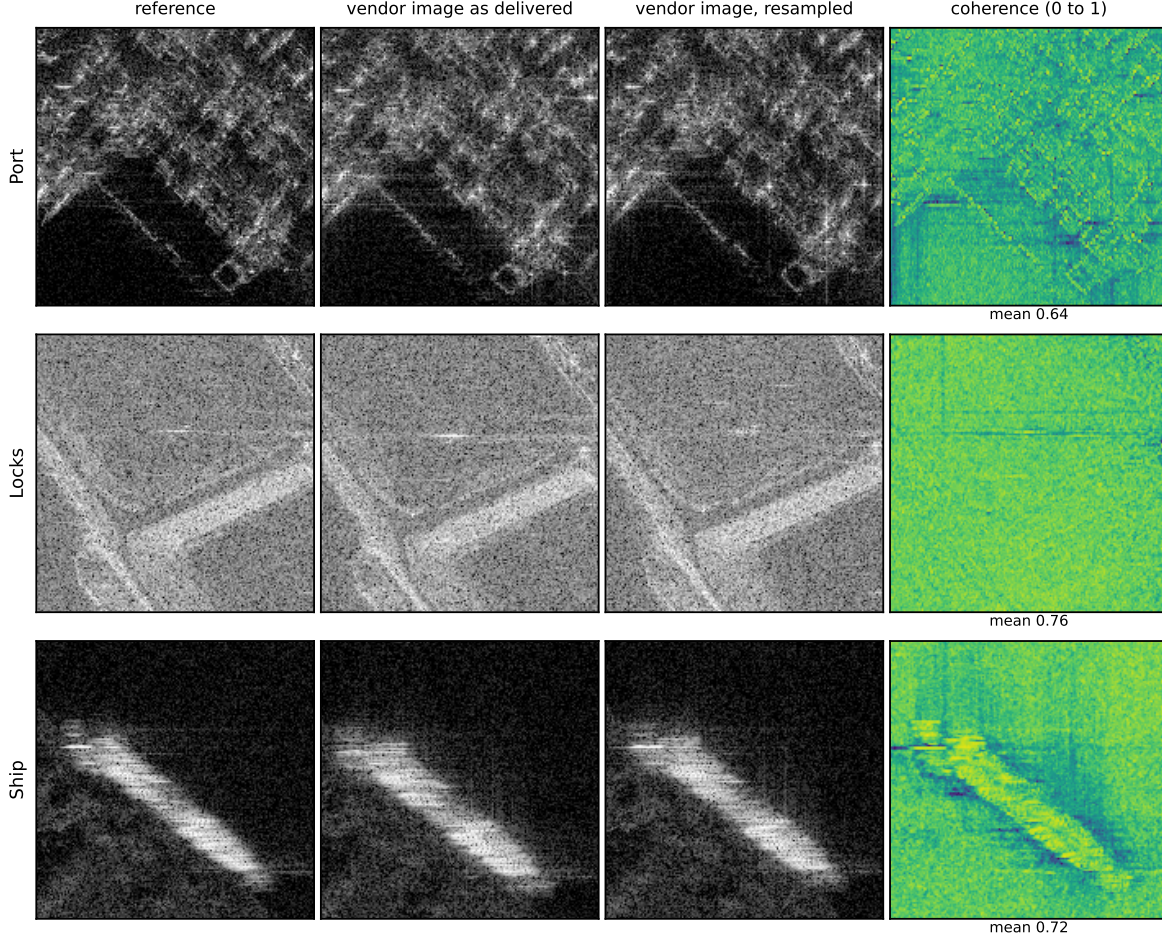


Figure 7: The port, lock and ship regions of Figure 3 (rows, 512 by 512 pixels). Columns: the reference; the vendor image as delivered, on the same pixels; the vendor image after resampling through the planar-wavefront displacement model of Appendix E; and the 5 by 5 coherence of the resampled vendor image with the reference after local re-alignment and phase-ramp removal, from 0 to 1, with its mean below each map. Amplitudes are displayed over a 45 dB range.

not expected. At the brightest isolated points of the lock and port regions (the ship is a moving target and the vegetated region has no isolated point) the impulse-response widths of the two images differ by 1 to 27%; the vendor’s is wider in azimuth at both points and narrower or equal in range, as a different aperture weighting would produce; the SICD metadata does not state whether an autofocus was applied, and the vendor’s motion compensation is independent of the one used here. A coherence of this order is therefore a property of the two processors and not a measure of the precision of either. The vendor image is delivered without a correction for the displacement; the polar-format images of this study remove it in the final resampling (Appendix E).

G Comparison with open-source implementations

With every CPU implementation on the c4d-highmem-16 instance of FastSAR’s CPU row and every GPU implementation on the same L4, FastSAR (<https://github.com/saulpingerman/FastSAR>)

Table 11: Error against the reference and time per full image on the Umbra Panama collection. Errors (dB, Section 5) span the three regions, worst to best (ISCE3: three 256 by 256 pixel patches, after removal of a common linear phase ramp); 0.0 dB corresponds to a difference with the energy of the reference. $16\times$, $8\times$ and $2\times$ give the oversampling of the range profiles. Times are measured, except those marked est., which scale the time on one region by the number of pixels and are upper bounds (the same scaling gives 2.4 h for FastSAR’s exact backprojection against 46 min measured).

implementation	machine	error (dB)	time, full image
RITSAR, as published, $16\times$	16 vCPUs (c4d)	−25.3 to −27.3	44 h (est.)
bpBasic , FFT 2^{17} (default)	16 vCPUs (c4d)	−38.1 to −43.2	57 h (est.)
bpBasic , FFT 2^{18}	16 vCPUs (c4d)	−52.9 to −55.4	69 h (est.)
ISCE3 backproject	16 vCPUs (c4d)	−31.5 to −54.9	4.8 h (est.)
FastSAR, exact, $8\times$	16 vCPUs (c4d)	−54.9 to −58.7	46 min
FastSAR, factorized, float32	16 vCPUs (c4d)	−55.8 to −61.2	12.4 s
ISCE3 backproject , CUDA	Nvidia L4	−31.5 to −52.2	27 min (est.)
torchbp, exact, $2\times$	Nvidia L4	0.0	15.2 s
torchbp, factorized	Nvidia L4	no image (zeros)	did not run (memory)
FastSAR, exact, $8\times$	Nvidia L4	−54.7 to −58.3	67 s
FastSAR, factorized, float32	Nvidia L4	−55.7 to −61.2	3.6 s
FastSAR, factorized, float16	Nvidia L4	−54.6 to −60.4	2.9 s

forms the Panama image in 12.4 s on the CPU and 2.9 to 3.6 s on the L4, at errors of −54.6 to −61.2 dB, against hours for the open-source backprojections (Table 11). With the algorithm held fixed, FastSAR’s exact backprojection takes 46 min on the CPU and 67 s on the L4, about 6 and 24 times less than ISCE3’s; on the CPU the factorized algorithm accounts for a further factor of about 220.

RITSAR [24] (commit 0e36d2e, ported to Python 3) is a NumPy library; **bpBasic** is the backprojection of the MATLAB toolbox of Gorham and Moore [27], written at the Air Force Research Laboratory and distributed by the National Geospatial-Intelligence Agency (commit 63f0401), run in GNU Octave 8.4; ISCE3 [25] (0.25.17) has C++/OpenMP and CUDA versions; torchbp [26] (commit b371884) is a PyTorch extension with CUDA kernels. Each received the Taylor-windowed phase history and antenna positions of the reference (Section 5) in its own input format and was scored with the error measure of that section on three of the regions of Figure 3, ISCE3 on 256 by 256 pixel patches of its own radar grid. Each had the whole instance; RITSAR and **bpBasic** do not parallelize their backprojection and kept about one core busy, ISCE3 about 5.6 cores. The scripts, records and a longer account of each comparison are in the directory **results/oss** of the study repository.

RITSAR’s error comes from two indexing defects, a range axis built with `linspace` whose spacing is too large by one part in $N_{\text{fft}} - 1$ and a frequency axis centered on the assumption of an even sample count, which the 14,399 samples of the collection do not satisfy. The error of **bpBasic** falls by 12 to 15 dB per doubling of its FFT length, as linear interpolation of the range profiles predicts, but stays 14 to 19 dB above that of exact backprojection at the same length, plausibly because it references the phase of its profiles to the lowest frequency of the band. ISCE3’s amplitude correlates with the reference at 0.9995 or more; its complex difference is mostly a linear phase ramp common to all patches, attributed to spectral centering, after whose removal the center patch agrees to −54.9 dB and the off-center patches keep an unexplained residual near −32 dB. torchbp evaluates the pixel-to-antenna distance in float32, whose resolution at the 744 km range of the collection is 6 cm, two wavelengths: its exact backprojection forms no image, while a float64 evaluation of the

Table 12: Polar format implementations on the Panama collection, on the c4d-highmem-16 instance: correlation of the log amplitude with the reference and mean coherence in 5 by 5 pixel windows over the three regions, and the time per full image.

implementation	log-amplitude correlation	coherence	time, full image
NGA <code>pfa_mem</code> (Octave)	0.80 to 0.89	0.76 to 0.88	44 s
FastSAR polar format, float32	0.996 to 0.999	0.999	11.3 s

same formula does, and its factorized backprojection needed more than 23 GiB, more than the L4 provides.

The NGA repository also holds a polar format, `pfa_mem`, run unmodified in Octave on the same instance (with a compatibility version of `cross` for mixed row and column inputs). Its image is not corrected for the planar-wavefront displacement, so each region was registered and resampled onto the reference pixels before scoring by amplitude statistics and coherence, as for the vendor image of Appendix F (Table 12); part of its coherence loss comes from that resampling.

H Measurement protocol in detail

Time and cost. Device time is measured from the start of the per-collection host work (for the factorized algorithm the float64 geometry of the first two levels, about 0.5 s) to the completion of the image on the device. Each configuration was compiled and run once and then timed once more; the second run is reported, and a few configurations were timed twice more, with a spread below 4%, so differences of that order between rows are not resolved.

The pipelined loop ran until at least four images had been formed and at least 40 s (polar format) or 60 s (factorized) had elapsed, whichever came later, except that the exact backprojection loops on the L4 ran two or three images (190 to 495 s) and polar format on the v6e two images (up to 215 s), and the reported figure is images divided by elapsed time. On the TPUs the pipelined loop overlaps the host work with the device and its throughput slightly exceeds the inverse of the device time; on the L4, whose host has four vCPUs, the factorized loops ran 6 to 11% slower per image than the device time.

The CPU instance’s utilization was recorded from `/proc/stat` during every run. The JAX program’s throughput was also measured with two and four worker processes pinned to disjoint cores, because one JAX process does not keep all 16 hardware threads busy; the C++ kernels run one process with one OpenMP thread pinned to each of the 8 cores, so the monitor reports half the hardware threads busy, and the energy estimate uses the core occupancy.

Power. The L4’s draw was sampled from `nvidia-smi` twice a second during the timed loops. The TPU virtual machines and the CPU instance expose no power counters; the bounds derived for them from published design figures are in Section 6.3 and are marked as estimates.

Precision. The gain fitted per algorithm family is a complex gain for the backprojections and a real one for polar format, whose phase differs from the reference by a smooth screen before the final resampling. Six statistics are computed for each image. The error is the energy of the difference relative to the energy of the reference (dB); this is the figure quoted when a single number is given. The others are the largest single-pixel difference relative to the brightest pixel (dB); the complex coherence between the two images over a 5 by 5 window, as the mean, the 0.1 percentile and the minimum; the amplitude ratio and the phase difference over the brighter half of the pixels, as the

99th percentile and the maximum; and the error in four rings by distance from the scene center. Appendix J tabulates all of them for every configuration whose image was saved. The selected configurations are defined in Section 5; the implementation forms among which the choice is made are the filter, ramp and resampling forms, and Appendices C and E give the measurements behind each choice.

CPU details. On the CPU the factorized float32 row comes from the C++ kernel build of Appendix D; the polar-format and float64 factorized rows come from the JAX program, timed as a single process. The reference was formed on a separate n2-highmem-48 instance (48 vCPUs, \$2.30 per hour) in 1892 s per Panama image, about \$1,210 per thousand images, and is not a candidate configuration.

Reference convergence and exact backprojection. The reference is a float64 exact back-projection with range profiles oversampled 16 times. Its convergence was measured on a 1024 by 1024 crop by forming the same crop at 8, 16 and 32 times: the 8 times image differs from the 32 times image by -55.5 dB and the 16 times image from the 32 times image by -69.2 dB. At 8 times the convergence error would have been comparable to the differences under study, so the 16 times reference was used; even so, no exact-backprojection error can be measured below the convergence of the reference.

With 16 times oversampled profiles, against the 16 times reference whose interpolation they share, the L4 float32 and float16 images measure -78.5 and -74.5 dB. At these settings the error of exact backprojection is set by the profile oversampling, approximately -56 dB at 8 times and, within the limit set by the convergence of the reference itself, approximately -69 dB at 16 times; float16 storage of the profiles adds no measurable error at 8 times, and at 16 times it separates the two shared-interpolation figures by 4 dB, which against the -69 dB floor amounts to less than 1 dB. With 16 times oversampling, exact backprojection takes 203.3 s with float16 profiles and 295.2 s with float32 profiles, which at the instance price is \$39.9 and \$58.0 per thousand images.

Polar format on each device. On the v5e the final resampling adds 94.3 s to the 3.1 s that precede it. The contiguous row gathers of the pulse resampling run nearly a hundred times faster on the TPU than the scattered two-dimensional gathers of the final resampling. A separable approximation of that step, expressible as row and column phase ramps, would avoid the gathers, but the squinted geometry of these collections is not exactly separable in this way. A form of the final resampling that avoids scattered gathers would reduce the TPU’s disadvantage in Table 3; the size of the reduction was not measured. The dense pulse-resampling product (images saved on the L4, the v6e and the CPU) and the gather form (the v5e and the CPU) give images that agree to the precision of Table 13.

I Ground truth of the Panama regions

The identity of the Panama regions used in Figure 3 was checked against Copernicus Sentinel-2 imagery [3]. Each pixel of the images formed here was located on the ground from the vendor’s SICD header: the pixel’s position in the slant plane follows from the scene center point and the row and column unit vectors, and the ground point is the intersection of the line through it along the slant plane’s normal with the WGS84 ellipsoid at the scene height. The image corners so computed agree with the corner coordinates in the SICD header to within 15 m. The nearest usable Sentinel-2 acquisition is from 2023-07-23, five days later, at 10 m resolution; a dry-season acquisition

of 2024-02-23 is shown as well because clouds cover part of the earlier one. The lock chambers, the quays and container yard of the terminal, and the road beside the vegetation region coincide in the two sensors. The bright target of the ship region, about 180 m long and close to the eastern bank, is not a fixed structure: the Panama Canal is a repeat-collection site in the Umbra catalog, and in the vendor’s geocoded products of six other passes, four of them three, four, five and seven days after this collection and two in February 2024, the same water is empty. The pass of 2023-07-25 02:30 UTC repeats the geometry of this collection to within 0.3° (azimuth 122.4° against 122.1° , grazing angle 42.6° in both, left-looking), which rules out an aspect-dependent return from a fixed object; the other passes span azimuths from -72° to 223° and grazing angles from 40° to 63° . The pass of 2023-07-23 14:47 UTC precedes the Sentinel-2 image of that day by 64 minutes, and neither shows anything at the position. The target is therefore a ship in the navigation channel; the Sentinel-2 image of 2024-02-23 shows a different ship nearby. The vendor’s geocoded product of this collection places the target inside the outlined crop, which confirms the pixel-to-ground mapping independently of the corner check.

The ship appears on the eastern bank, inland of the channel, as a consequence of its motion during the aperture. A stationary-scene processor places a target with line-of-sight velocity v_r at the position whose Doppler history matches, displaced along azimuth by about Rv_r/V (here about 100 m per m/s), and the along-track velocity defocuses it [2]. Since exact backprojection takes the antenna position of every pulse as data, a target moving at constant velocity $\mathbf{v}$ is formed exactly by backprojecting with the track shifted by $-\mathbf{v}t$, which both focuses it and places it at its position at the aperture center. The ship was assumed to follow the channel axis (bearing 158° , from the Sentinel-2 water mask) and its speed was swept from -8 to 8 m/s in a tile around it (Figure 8). The sharpness of the tile, the sum of the squared intensities, is single-peaked at 3.0 m/s (5.8 knots toward the Pacific), where the peak intensity is 10.6 dB above the stationary-scene image and the ship has moved 469 pixels (194 m) along azimuth into the navigation channel; the position of the water was not used in the estimate. A heading 10° to the west (148°) gives the same speed with an 18% lower sharpness peak and a compensated position 22 m away. The images compared in Figures 3 and 4 are the stationary-scene images, in which the ship appears at the same displaced position under every algorithm; the motion analysis serves only to establish that the target is a ship.

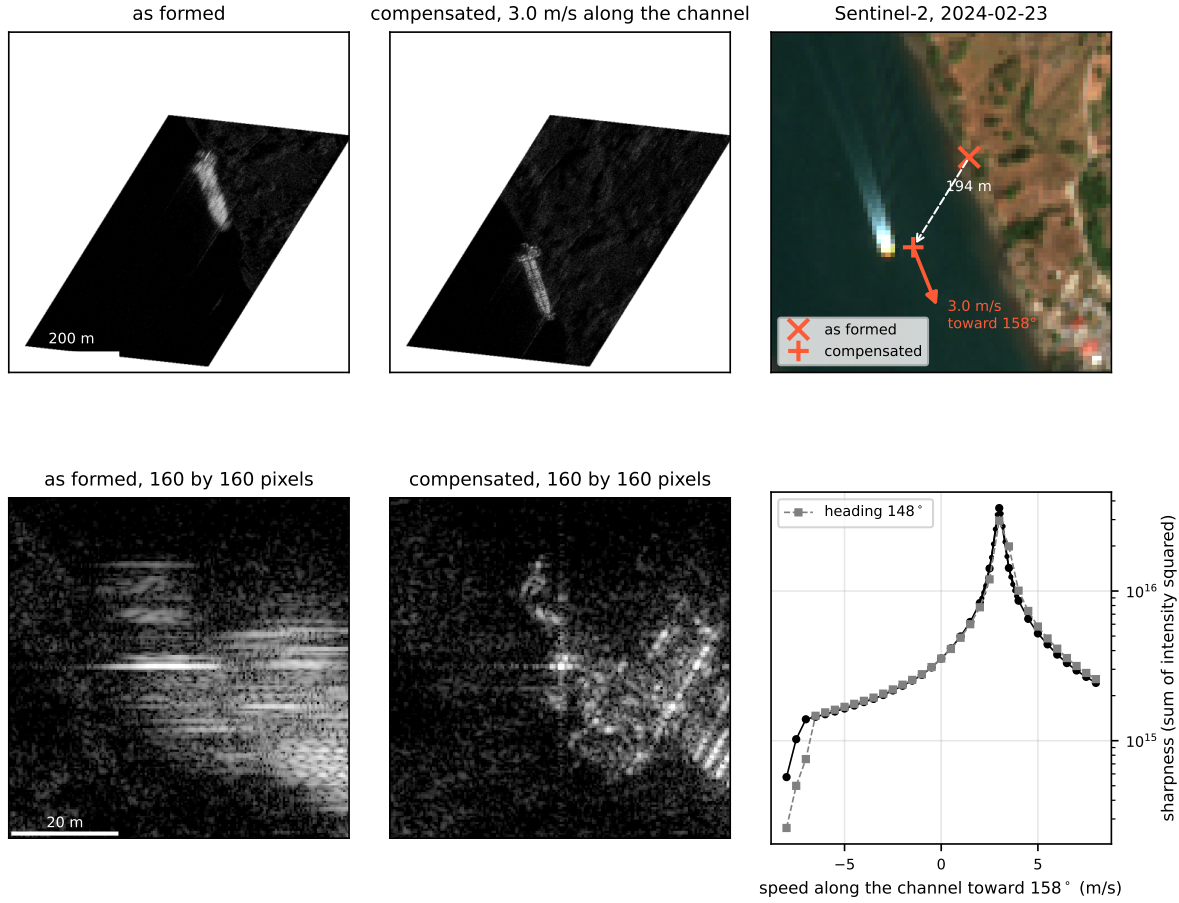


Figure 8: The ship compensated for its motion. Top: the tile around the ship as formed and after backprojection with the antenna track shifted for a ship moving at 3.0 m/s along the channel, both resampled north up over a 760 m window (200 m bar), and the two positions on the Sentinel-2 image of 2024-02-23, with the 194 m displacement (dashed) and the estimated velocity, 3.0 m/s toward 158°, drawn at the compensated position. Bottom: 160 by 160 pixel windows around the ship in image coordinates (20 m bar), and the sharpness of the tile against the assumed speed for the channel heading of 158° (coarse and fine sweeps) and for 148°.

Table 13: Precision of the Panama Canal images against the float64 exact backprojection: energy of the difference (dB), largest pixel difference relative to the brightest pixel (dB), minimum 5 by 5 coherence (the mean and 0.1 percentile are in Appendix J), amplitude ratio (99th percentile and maximum, dB) and phase difference (99th percentile and maximum, degrees) over the brighter half of the pixels. The two 16 times oversampled rows share the reference interpolation and read better than their estimated true error of about -69 dB (Section 6.2). The L4 float32 row is the dense-filter image; the convolution form applies the same filter coefficients. The ramp form does not alter the image; TPU rows are labeled with the form whose image was saved.

Device	Configuration	Error	Max	Coh. min	Amp. p99	max	Phase p99	max
L4	Exact BP, float32 profiles	-56.9	-62.2	0.923	0.03	2.5	0.2	15
L4	Exact BP, float16 profiles	-56.8	-62.2	0.922	0.03	2.5	0.2	15
L4	Exact BP, float32 profiles, 16x oversampled	-78.5	-85.5	1.000	0.00	0.1	0.0	0
L4	Exact BP, float16 profiles, 16x oversampled	-74.5	-85.2	1.000	0.01	0.1	0.0	1
L4	Factorized BP, float32, CUDA kernels	-59.5	-64.7	0.886	0.03	3.2	0.2	18
L4	Factorized BP, float16 throughout, CUDA kernels	-58.6	-64.4	0.886	0.03	3.2	0.2	18
L4	Factorized BP, float32, float16 final stage, CUDA kernels	-59.3	-64.4	0.886	0.03	3.2	0.2	18
L4	Factorized BP, float32	-57.3	-64.3	0.886	0.03	3.4	0.2	18
L4	Factorized BP, TF32 products	-56.8	-65.0	0.842	0.04	3.2	0.2	20
L4	Factorized BP, float16 products, conv. filters	-56.2	-64.2	0.843	0.04	3.2	0.3	20
TPU v6e	Factorized BP, three-pass products, fused kernels	-57.5	-64.2	0.886	0.03	3.6	0.2	18
TPU v6e	Factorized BP, single-pass products, fused kernels	-45.8	-54.1	0.705	0.16	10.1	1.0	86
TPU v5e	Factorized BP, three-pass products, fused kernels	-57.6	-64.5	0.886	0.03	3.4	0.2	18
TPU v5e	Factorized BP, single-pass products, fused kernels	-45.8	-54.2	0.705	0.16	9.4	1.0	83
TPU v6e	Factorized BP, six-pass products	-57.5	-64.2	0.886	0.03	3.6	0.2	18
TPU v6e	Factorized BP, three-pass products, direct ramps	-57.5	-64.2	0.884	0.03	3.6	0.2	18
TPU v6e	Factorized BP, single-pass products	-45.8	-54.1	0.707	0.16	9.9	1.0	86
TPU v6e	Factorized BP, single-pass, bfloat16 storage	-45.8	-54.1	0.707	0.16	9.9	1.0	86
TPU v5e	Factorized BP, three-pass products, direct ramps	-57.6	-64.5	0.884	0.03	3.3	0.2	18
TPU v5e	Factorized BP, single-pass products	-45.8	-54.2	0.707	0.16	9.3	1.0	81
CPU	Factorized BP, float64, conv. filters, direct ramps	-59.6	-64.6	0.886	0.03	3.2	0.2	18
CPU	Factorized BP, float32, C++ kernels	-59.6	-64.6	0.886	0.03	3.2	0.2	18
CPU	Factorized BP, float32, conv. filters, direct ramps	-58.0	-64.4	0.886	0.03	3.2	0.2	18
TPU v6e	Polar format, float32	-32.1	-40.1	0.680	0.69	20.7	6.1	70
L4	Polar format, float32	-32.1	-40.1	0.680	0.69	20.7	6.1	70
CPU	Polar format, float32, gather	-32.1	-40.2	0.679	0.69	20.1	6.1	70

J Precision and cost tables

Table 13 gives the error statistics of the selected Panama configurations and Table 14 the cost figures for Melbourne and Iowa; the device time and statistics of every configuration timed or measured on each scene are in the supplementary tables of the study repository ([report/v2/supplement_tables.pdf](#)). Configurations are labeled as in the body: on the TPUs *single-pass*, *three-pass* and *six-pass products* are the one, three and six bfloat16 passes of a float32 product and *bfloat16 storage* keeps the data in bfloat16; on the L4 *TF32 products* is the product at JAX’s default precision and *float32* the product at the highest setting, used for every float32 figure in the body; *gather* marks the gather form of the pulse resampling and of the final resampling; *direct ramps*, *conv. filters* and *tap sums* are the forms described in Appendix A.

Table 14: Melbourne and Iowa: device time, throughput and cost for the selected configurations, as in Table 3; bold marks the fastest configuration on each collection.

Device	Algorithm	Arithmetic	Time (s)	Images/h	\$ per 1000	kWh per 1000
<i>Melbourne</i>						
L4	Exact BP	float16 profiles	102.7	35	19.99	2.02
L4	Exact BP	float32 profiles	248.2	15	48.58	4.94
L4	Factorized BP	float16 throughout, CUDA kernels	2.9	1,128	0.63	0.06
L4	Factorized BP	float32, CUDA kernels	3.6	911	0.78	0.07
L4	Polar format	float32, gather	2.2	1,562	0.45	0.04
TPU v5e	Factorized BP	single-pass products, fused kernels	2.7	1,227	0.98	0.10 to 0.16
TPU v5e	Factorized BP	three-pass products, fused kernels	4.7	726	1.65	0.17 to 0.28
TPU v5e	Polar format	float32, gather	62.5	58*	20.84	2.08 to 3.47
TPU v6e	Factorized BP	single-pass products, fused kernels	1.6	2,092	1.29	0.10 to 0.17
TPU v6e	Factorized BP	three-pass products, fused kernels	2.1	1,562	1.73	0.13 to 0.22
TPU v6e	Polar format	float32	101.7	35	77.96	5.78 to 10.11
CPU	Factorized BP	float32, C++ kernels	12.5	293	3.29	0.08
CPU	Factorized BP	float64, conv. filters, direct ramps	642.0	6*	171.90	2.92
CPU	Polar format	float32, gather	11.6	310*	3.11	0.08
<i>Iowa farmland</i>						
L4	Exact BP	float16 profiles	63.8	57	12.46	1.26
L4	Exact BP	float32 profiles	142.3	25	27.85	2.83
L4	Factorized BP	float16 throughout, CUDA kernels	1.8	1,784	0.40	0.04
L4	Factorized BP	float32, CUDA kernels	2.3	1,424	0.50	0.05
L4	Polar format	float32, gather	1.6	2,108	0.34	0.03
TPU v5e	Factorized BP	single-pass products, fused kernels	1.8	1,824	0.66	0.07 to 0.11
TPU v5e	Factorized BP	three-pass products, fused kernels	3.0	1,122	1.07	0.11 to 0.18
TPU v5e	Polar format	float32, gather	44.2	81*	14.74	1.47 to 2.46
TPU v6e	Factorized BP	single-pass products, fused kernels	1.1	2,948	0.92	0.07 to 0.12
TPU v6e	Factorized BP	three-pass products, fused kernels	1.6	2,139	1.26	0.09 to 0.16
TPU v6e	Polar format	float32	57.8	61	44.49	3.30 to 5.77
CPU	Factorized BP	float32, C++ kernels	8.5	425	2.27	0.06
CPU	Factorized BP	float64, conv. filters, direct ramps	409.9	9*	109.75	1.63
CPU	Polar format	float32, gather	8.2	440*	2.19	0.06